



DEDICATION

THIS WORK IS DEDICATED
TO MY HARDWORK



ACKNOWLEDGEMENTS

The accomplishment of this great project won't have been a success without out the important contributions of a good number of people who gave me a huge support academically, professionally, financially and even spiritually. I then direct my appreciations to:

- Mr. Armand Claude ABANDA, the Resident Representative of AICS Cameroon for creating the Anglophone section offering the software engineering field and also for his encouragements and follow-ups. Also, for permitting us carry our internship at his institute AICS Cameroon.
- My Academic Supervisor Mr. TCHOUA NGATCHOU NANA Samory for his time, advices, and follow-up throughout these periods.
- My Professional Supervisor Mr. Bangoup Ruphin Samory for his time, advices, and follow-up throughout these periods
- To my teachers (lecturers) for their words of encouragement, trust and time invested during this period.
- To my lovely family YOGO, my mum and dad must especially, thanking them for their continuous prayers, blessings, support (spiritual and financial support) and encouragements. A special thanks to my uncles and aunts.
- To my elder sister, for her support and advices



TABLE OF CONTENT

DEDICATION	I
ACKNOWLEDGEMENTS.....	II
TABLE OF CONTENT	III
LIST OF FIGURES.....	IV
LIST OF TABLES.....	VI
LIST OF ABBREVIATIONS	VII
ABSTRACT.....	1
RÉSUMÉ	2
GENERAL INTRODUCTION.....	3
CHAPTER I : INSERTION PHASE.....	5
CHAPTER II:	18
EXISTING SYSTEM.....	18
CHAPTER III :	28
CHAPTER IV :	52
ANALYSIS PHASE	52
CHAPTER V :	86
CONCEPTION PHASE.....	86
CHAPTER VI: REALIZATION PHASE	102
CHAPTER VII:	115
FUNCTIONALITY TESTING.....	115
CHAPTER VIII:	122
INSTALLATION GUIDE AND USER GUIDE	122
GENERAL CONCLUSION	132
BIBLIOGRAPHY	133
WEBOGRAPHY	134
ANNEXES.....	136

LIST OF FIGURES

Figure 1: Localisation Plan of INTERLINK	9
Figure 2 : Organisation chart of INTERLINK	12
Figure 3: Gantt chart	49
Figure 4: UML hierarchy.....	56
Figure 5: 2TUP representation	59
Figure 6: Formalism of use case	62
Figure 7: General use case diagram	64
Figure 8:manage security specific use case	65
Figure 9: Manage residents specific use case.....	65
Figure 10: Light control specific use case	66
Figure 11: Formalism of communication diagram.....	69
Figure 12:Communication diagram for Authentication.....	71
Figure 13:Activate security communication diagram.....	72
Figure 14:add resident communication diagrams	73
Figure 15:Formalism of sequence diagram	74
Figure 16: Activate security sequence diagram.....	76
Figure 17: Sequence diagram for Authentication.....	77
Figure 18:Sequence diagram for authentication.....	78
Figure 19: Add resident sequence diagram	79
Figure 20:Formalism of activity diagram	80
Figure 21:Activity diagram for authentication	82
Figure 22:Activity diagram fo adding residents.....	83
Figure 23:activity diagram for activating security	84
Figure 24: n-tier architecture	89
Figure 25:MVVM architecture.....	90
Figure 26: Formalism of class diagram	92
Figure 27: Class diagram for our app	94
Figure 28: formalism of a state machine diagram.....	95



Figure 29: State machine for payment	97
Figure 30: Formalism of package diagram.....	98
Figure 31: Package diagram for system.....	100
Figure 32: Formalism of deployment diagram.....	105
Figure 33: System's deployment diagram.....	107
Figure 34: Formalism of component diagram	108
Figure 35: component diagram of the system	110
Figure 36: Authentication screen	117
Figure 37: Add resident screen.....	118
Figure 38: Smart light screen	119
Figure 39: Login test	120
Figure 40: Add resident test	121
Figure 41: Dashboard screen	127
Figure 42: Monitoring security screen.....	128
Figure 43: Monitoring devices screen	130



LIST OF TABLES

Table 1: Partners of INTERLINK	11
Table 2: Company hardware resources	13
Table 3: Company software resources	14
Table 4: Limitation of the existing system and proposed solutions	24
Table 5: Software estimated cost	42
Table 6: Hardware estimated cost	44
Table 7: Human resources estimated cost	45
Table 8: Total cost	45
Table 9: Project planning	46
Table 10: Participants	48
Table 11: Comparing between MERISE and UML	57
Table 12: Components of a use case diagram	62
Table 13: Description of manage residents use case	66
Table 14: manage security use case description	67
Table 15: Components of a communication diagram	69
Table 16: Components of a sequence diagram	74
Table 17: Components of an activity diagram	81
Table 18: Components of a class diagram	92
Table 19: Components of a state machine diagram	96
Table 20: Components of package diagram	98
Table 21: Components of a deployment diagram	106
Table 22: Components of a component diagram	108
Table 23: Hardware material for development	111
Table 24: Software used for development	112



LIST OF ABBREVIATIONS

2TUP: Two Track Unified Process

API: Application programming Interface

AICS: African institute of Computer Science

CD-ROM: Compact Disk- Read Only Memory

CRUD: Create Read Update Delete;

DBMS: Database Management System

ICT: Information and Communication Technology

IOT: Internet of things

SDLC: System Development Life Cycle

ABSTRACT

Humanity has stopped evolving. In fact, it has evolved at an exponential rate over the past few centuries, driven by innovations in technology. From the agricultural revolution to the industrial revolution then to information and communication technology revolution, we have developed new tools that have allowed us to transform our world in ways that were unimaginable. Thus, we cannot see the future without technology. These new technological advances have had a profound impact on our lives, changing the way we work, the way we communicate and the way we interact with the world around us. For this reason, the African Institute of computer sciences is committed to making Cameroon a key player in the information and communication technology revolution. To achieve this, AICS offers its students practical trainings through academic internship. This internship gives students the opportunity to apply their theoretical knowledge to real world problems. In line with these perspectives, we approached INTERLINK, which is an enterprise that hosts our internship. During our induction phase, we were assigned the theme” **Design and Implementation of an Intelligent House System Based on Light Intensity Control and Security Monitoring** “. Following our theme, we conducted feasibility studies, followed by analysis, then via conception and realization phases. The application was developed using Nodejs, Vue.js and in order to obtain a flexible and secure solution.

Keywords:

- ARTIFICIAL INTELLIGENCE
- IOT
- SCHEDULING
- COMMUNICATION

RÉSUMÉ

L'humanité n'a pas cessé d'évoluer, bien au contraire, elle a évolué à un rythme exponentiel au cours des derniers siècles, stimulée par les innovations technologiques. De la révolution agricole à la révolution industrielle, puis à la révolution des technologies de l'information et de la communication, nous avons développé de nouveaux outils qui nous ont permis de transformer notre monde de manière inimaginable. Ainsi, nous ne pouvons pas concevoir l'avenir sans technologie. Ces avancées technologiques ont eu un impact profond sur nos vies, changeant la façon dont nous travaillons, communiquons et interagissons avec le monde qui nous entoure. Pour cette raison, l'Institut Africain d'Informatique (IAI) s'est engagé à faire du Cameroun un acteur clé de la révolution des technologies de l'information et de la communication. Pour atteindre cet objectif, l'IAI offre à ses étudiants des formations pratiques à travers des stages académiques. Ce stage donne aux étudiants l'opportunité d'appliquer leurs connaissances théoriques à des problèmes du monde réel. Dans cette optique, nous nous sommes adressés à SMART-TECH, une entreprise qui accueille notre stage. Lors de notre phase d'induction, nous avons été assignés au thème " **Conception et mise en œuvre d'un système de maison intelligente basée sur le control de luminosité et surveillance** ". Suivant notre thème, nous avons effectué des études de faisabilité, suivies d'une analyse, puis des phases de conception et de réalisation. L'application a été développée en utilisant Nodejs, Vue.js afin d'obtenir une solution flexible et sécurisée.

Mots-clés :

- Intelligence artificielle
- Internet des Object
- Planification
- communication

GENERAL INTRODUCTION

In today's residential environment, homeowners face growing challenges in managing household security and energy consumption efficiently while ensuring the safety and comfort of residents. Traditional home security and lighting systems often operate independently and rely heavily on manual monitoring, providing limited real-time awareness and little intelligent response to abnormal situations. This gap can lead to delayed detection of security threats, unnecessary energy consumption, and limited control over household devices. To overcome these limitations, this project presents the **conception and implementation of an intelligent house system based on light intensity control and security monitoring**, which combines smart lighting, security monitoring, and artificial intelligence to improve household safety and energy efficiency. The system integrates sensors, cameras, smart lighting devices, facial recognition, and AI-based threat analysis to monitor the home environment and assist in identifying potentially dangerous situations. It also provides residents with centralized control and real-time notifications, while supporting automated and manual management of connected devices. The project's analysis and design adopt the **2TUP (Two-Track Unified Process)** methodology, emphasizing both business and system modeling to obtain a comprehensive understanding of user needs and system functionalities. Using **UML (Unified Modeling Language)** diagrams—such as use case, class, sequence, activity, and state machine diagrams—the system is modeled to represent its behavior, user interactions, processes, and data structures. This structured analytical approach establishes a solid foundation for the subsequent design, implementation, and testing phases of the intelligent house system.

Project Phases:

- ✓ **Insertion Phase:** Defines project context, objectives, and overall purpose.
- ✓ **Existing System:** Manual, slow, decentralized, and prone to information gaps.
- ✓ **Specification Book:** Lists system requirements and development guidelines.
- ✓ **Analysis Phase:** Identifies user needs and system functionalities.
- ✓ **Conception Phase:** Designs system architecture and component interactions.
- ✓ **Realization Phase:** Develops and integrates the designed system modules.



Design and Implementation of an Intelligent House System Based on Light Intensity Control and Security Monitoring.



- ✓ **Functionality Testing Phase:** Verifies system performance and requirement compliance.
- ✓ **The user guide:** Explains registration, navigation, and application process clearly



CHAPTER I : INSERTION PHASE

Preamble

The insertion phase document presents the internship environment, the structure of the organization, the conditions in which the interns were received, as well as the project to be realized during the internship.

Content :

INTRODUCTION

- I. WELCOME AND INTERGRATION
- II. GENERAL PRESENTATION OF SMART-TECH
- III. HARDWARE AND SOFTWARE RESOURCES OF WELLD OF SMART-TECH
- IV. ORGANIZATION OF SMART-TECH
- V. BRIEF PRESENTATION OF PROJECT

CONCLUSION



INTRODUCTION

Social and professional integration refers to the process by which an individual is integrated into the social and economic context by adopting the norms and rules that govern it. In the following sections, we will begin by describing our reception and integration within the company **INTERLINK**. Then, we will explain its overall structure as well as its organizational chart.



WELCOMING AND INTERGRATION

Welcome

Upon our arrival at INTERLINK, the reception was very simple. We were first taken to the meeting room, introduced to each team member, and then each of us introduced ourselves with our name, class, school, reason for being there, duration of the internship, and how we were admitted.

The Director then gave a welcome speech and instructed the intern coordinator to give us a full tour of the company. Finally, based on our academic fields, we were assigned to different departments

Integration

At first, adjusting was quite intense due to the workload and pressure of meeting tight deadlines. However, after three days, adaptation was achieved.

GENERAL PRESENTATION OF INTERLINK

1. History

INTERLINK is a company specializing in software development, online presence management, digital marketing, tech consulting, systems maintenance, IT training and consultancy, network design and implementation, and surveillance camera installation.

It was founded in 2025 by Mr. TCHOU NGATCHOU NANA Samory and Mr. BANGOUP Rufin, who had a futuristic vision for digital transformation in businesses. Today,

INTERLINK supports and works with over 10 companies aiming to align with digital standards. Its headquarters is in Yaounde, with a branch office in Douala.

2. Geographical location

INTERLINK is located at Abang, 3 kilo meters from AWAE Escalier.

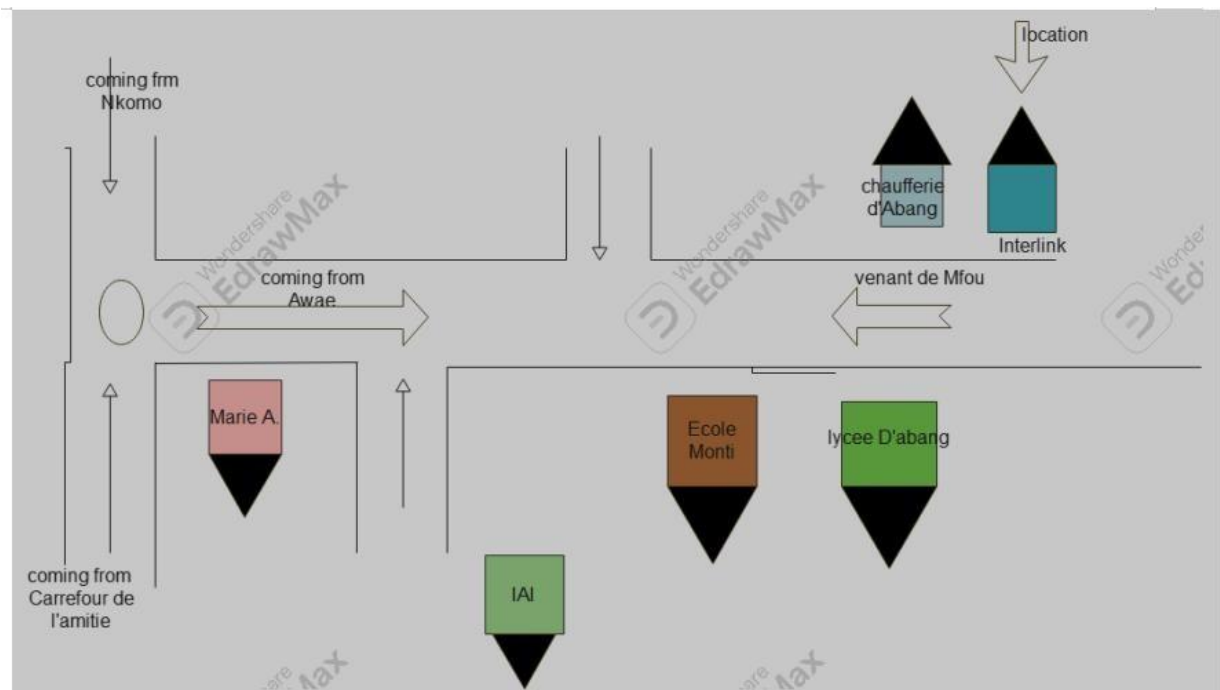


Figure 1: Localisation Plan of INTERLINK

3. Mission

INTERLINK main mission is to promote digital transformation for individuals and businesses. Its objectives include:

- Providing a new image of Africa (especially Cameroon) to the world by helping businesses move from analog to digital to improve performance.
- Creating software, logos, flyers, websites, and network installations.
- Offering quality training to company staff and individuals, believing every African should master digital tools.

Activities, Achievements and Partners.

ACTIVITIES

INTERLINK is a key player in digital transformation across various business sectors. Its solutions include:

- ✧ System Integration: Helping companies adapt to digital tools.
- ✧ IT Strategy and Organization Consulting: Advising on IT service organization and strategies.
- ✧ Project Management: Supporting businesses throughout project execution.
- ✧ Communication & Digital Marketing: Handling online presence, including web design and content.
- ✧ Web & Mobile Application Development: Building websites and apps with attention to quality.

ACHIEVEMENTS

- Design and implementation of a Travel Management System (TMS) for GHF SARL
- Network deployment and maintenance for SOCADEP, CWORDH ENERGY, and individuals
- Installation of surveillance systems at EMERIX COMPANY and private residences
- Design of logos and flyers for SMARTEKS, LA PETITE BAYAM, FONDATION PAVI, and individuals
- Online presencemanagement for LA PETITE BAYAM, SMARTEKS, PENTIUM GSM, and others
- Creation of a professional training center with JPTEKS
- Maintenance of IT equipment for FONDATION PAVI and others.

PARTNERS

INTERLINK as an economic enterprise has the following partners who greatly help in the development of their enterprise.

Table 1: Partners of INTERLINK

PARTNERS	
□ SOCADEP	
□ EMERIX.	
□ JPTEKS	  

□ PENTUIM GSM



□ FONDATION PAVI.

ORGANIZATIONAL CHART OF SMART-TECH

SMART-TECH has several departments. We were assigned to the Technology Department.

Figure 2 : Organisation chart of INTERLINK

COMPANY RESOURCES

1. Hardware Resources

Table 2: Company hardware resources

N	Equipment	Characteristic	Quantity
1.	Computer sever	Desktop HP 	03
2.	Laptop computer	Laptop HP 	05
		Laptop Dell 	
3.	Routers	Cisco catalyst 1941 	01
		Tp-link (wifi) 	01
4.	Switch	Switch cisco catalyst 24 ports 	01
5.	Optic fibre	Camtel	01
6.	Modem	Modems MTN 4G 01	01
		Modems Toplink 5G	01

7.	Ethernet cable	Twisted pair CAT6	01
----	----------------	-------------------	----

2. Software Resources

Table 3: Company software resources

N°	Software	Description
01	 Office suite	Standard Office Suite for creating documents, spreadsheets, presentations, and other professional content.
02	 Windows 11	Operating System for computer.
03	 WPS	Word Processing software.
04	 Adobe suit	Professional Suite for image editing, illustration creation, video editing, and audio editing.

05	 Canva	Online Graphic Design Platform for creating professional visuals, presentations, social media publications, etc.
06	 Visual studio code	IDE (Integrated Development Environment) , a free and open-source environment, very popular for web and mobile development.
07	 Wamp	Local Web Development Environments , including Apache, MySQL, and PHP, for testing and deploying web applications locally before putting them online.
08	 Cisco ISE	Authentication Management Tools (such as Active Directory) for managing authentications within information systems
09	 ENSP of HUAWEI	Email Monitoring Software (such as Microsoft Exchange Monitor), a useful tool for monitoring and optimizing mail server operations
10	 SolarWinds	Email Monitoring Software (such as Microsoft Exchange Monitor), a useful tool for monitoring and optimizing mail server operations

11	 EVE NG	GNS3 (or other network simulation software), a software for simulating networks through clientless virtualization.
12	 Eclipse	Open-source Integrated Development Environment (IDE) Used for software development in various languages, including:Java, C++, Python, PHP
	 GitHub	Source Code Management Platform , used to store, share, and collaborate on software projects.



CONCLUSION

Upon completing our integration, we have gained a thorough understanding of the work environment in which we will be operating during our internship. We have acquainted ourselves with the structure's history, missions, responsibilities, organization, various resources, and organizational chart, enabling us to confidently proceed to the second part of the report, the specifications document.



CHAPTER II: EXISTING SYSTEM



Preamble:

The insertion phase helped us to establish a comprehensive understanding of the host organization. This phase emphasizes on the key characteristics, specificities, and objectives of the designated study subject

Content :

INTRODUCTION

- I. PRESENTATION OF THEME
- II. DESCRIPTION OF THE EXISTING SYSTEM
- III. CRITICISM OF THE EXISTING SYSTEM
- IV. LIMITATIONS OF EXISTING SYSTEM
- V. PROBLEMATIC
- VI. PROPOSED SOLUTION

CONCLUSION



INTRODUCTION

Engineers are society's problem-solvers, working to make life easier for everyone. Before proposing solutions, we need to understand the root causes of the problem and how it's currently addressed. Analysing existing systems in our field is key to identifying problems and developing effective solutions. This ensures our proposals improve upon or replace the current system for the better

PRESENTATION OF THE THEME

Modern homes increasingly require better solutions for security, energy efficiency, and convenient management of household devices. Traditional houses mainly rely on manual control of lighting and human intervention for security monitoring. This can lead to energy wastage, delayed detection of suspicious activities, and limited ability to respond quickly to security threats.

To address these challenges, this project proposes the development of an Intelligent House System that combines Internet of Things (IoT), Artificial Intelligence (AI), facial recognition, motion detection, smart lighting, and automated security monitoring.

The system will allow homeowners and authorized residents to remotely monitor and manage their house through a mobile application. It will automatically control lighting according to environmental conditions and provide security monitoring through cameras and sensors.

Furthermore, the AI component will analyze detected activities and facial information to distinguish normal situations from potential security threats. When a genuine security threat is confirmed, the system can automatically generate an alert and initiate communication with the appropriate security service or police.

The proposed solution therefore aims to improve home security, energy efficiency, responsiveness, and convenience while providing centralized and secure management of connected devices.

THEME :

Conception and Implementation of an Intelligent House System Based on Smart Lighting and AI-Powered Security Monitoring

DEFINITION OF TERMS

Artificial Intelligence (AI)

Artificial Intelligence refers to the capability of a computer system to analyze data, recognize patterns, and make decisions or predictions based on that information.

Intelligent House System

An Intelligent House System is a software and hardware platform that allows household devices and security components to be monitored, controlled, and automated.

Internet of Things (IoT)

IoT refers to the connection of physical devices such as sensors, cameras, lights, and microcontrollers to a network so that they can exchange information and be controlled digitally.

Facial Recognition

Facial recognition is an AI-based technology used to identify or verify a person by analyzing facial characteristics captured by a camera.

Threat Analysis

Threat analysis is the process of evaluating information collected from sensors and cameras to determine whether an observed situation represents a potential security threat.

Smart Lighting

Smart lighting is a lighting system that can be automatically or remotely controlled, including switching lights on/off and adjusting their brightness according to environmental conditions or user commands.

DESCRIPTION OF THE EXISTING SYSTEM

Traditional residential security and lighting systems are generally fragmented and require significant human intervention. The existing situation can be characterized by:

- **Manual lighting control:** Lights are generally switched on and off manually, which can result in unnecessary energy consumption.
- **Limited security monitoring:** Traditional homes may depend on basic locks, alarms, or cameras without intelligent analysis of the collected information.
- **No intelligent facial recognition:** Conventional security systems cannot automatically distinguish authorized residents from unknown individuals.
- **Delayed threat detection:** Suspicious activities may only be noticed after a person observes a camera or receives an alarm.
- **No automated threat analysis:** Existing systems generally do not combine information from cameras and sensors to determine whether a situation represents a genuine threat.
- **Limited remote control:** Homeowners may not have a centralized mobile application for managing their household devices.
- **Fragmented devices:** Cameras, lights, sensors, and security systems may operate independently rather than as one integrated platform.
- **No intelligent lighting adjustment:** Conventional lighting systems do not automatically adapt brightness according to ambient light conditions.
- **Manual emergency response:** In many situations, the homeowner or resident must manually contact emergency services.
- **Limited activity history:** Security events and device activities may not be centrally recorded for later consultation.

These limitations demonstrate the need for an integrated intelligent house system capable of combining **IoT, AI, smart lighting, and security monitoring**.

CRITICISM OF THE EXISTING SYSTEM

The traditional approach to residential management presents several important shortcomings:

1. Manual lighting management

Users must manually control lights, which can cause unnecessary energy consumption.

2. Lack of intelligent security

Traditional systems detect events but do not necessarily understand or analyze them.

3. No facial recognition

There is no automated mechanism for identifying authorized and unknown persons.

4. Delayed threat response

A security incident may require human observation before action is taken.

5. Lack of centralized management

Different household devices may be controlled through separate systems.

6. Limited remote accessibility

Homeowners may not be able to monitor and control their house remotely through a single application.

7. Lack of automatic emergency response

The user normally has to manually contact the appropriate security service.

8. Lack of intelligent lighting

Brightness is generally not automatically adjusted according to environmental conditions.

9. Poor event tracking

Security incidents and device activities may not be properly recorded.

10. Security and privacy concerns

Without proper authentication and authorization, unauthorized users could potentially access household devices or information.

LIMITATIONS OF THE EXISTING SYSTEM AND PROPOSED SOLUTIONS

Table 4: Limitation of the existing system and proposed solutions

Limitation	Consequence	Proposed Solution
Manual lighting control	Energy wastage	Automatic smart lighting

No intelligent monitoring	Suspicious activities may go unnoticed	AI-powered security monitoring
No facial recognition	Difficult to identify persons	Facial recognition
Delayed threat detection	Slow response to incidents	AI threat analysis
Manual emergency response	Delayed contact with security services	Automated emergency notification
Fragmented devices	Difficult management	Centralized mobile application
Limited remote access	Homeowner cannot easily monitor the house	Flutter mobile application
No brightness automation	Inefficient lighting	LDR-based brightness adjustment
Poor event tracking	Difficult to investigate incidents	Security event history
Weak access control	Unauthorized device access	Authentication and role-based authorization
Multiple houses not isolated	Risk of cross-house access	House-based data and device isolation

PROBLEMATIC

How can a residential environment improve security, lighting efficiency, and emergency response through the integration of IoT devices, Artificial Intelligence, facial recognition, and a centralized mobile application while ensuring secure isolation between different houses?

PROPOSED SOLUTION

To address the limitations of traditional residential systems, this project proposes the development of an **Intelligent House System** integrating IoT, Artificial Intelligence, smart lighting, facial recognition, and security monitoring.

The system will provide a centralized mobile application developed with **Flutter** through which authorized homeowners and residents can interact with the house.

The backend will be developed using **Node.js and Express.js**, with **Sequelize** serving as the ORM and **MySQL hosted through XAMPP** serving as the database.

The system will integrate devices such as:

- ESP32
- ESP32-CAM
- Motion sensors
- LDR/light sensors
- Smart lights
- Cameras

The security subsystem will operate as follows:

Motion detection → Camera activation → Person detection → Facial recognition → AI threat analysis → Threat verification → Alert/emergency response

The lighting subsystem will provide:

Ambient light detection → Brightness calculation → ESP32 → Smart light

The system will also support a **manual emergency button** that can be activated by the homeowner or an authorized resident.

For automatically detected threats, the AI system can trigger the emergency workflow once the system has sufficiently confirmed that the situation represents a genuine security threat.

Finally, because the platform can manage multiple houses, every house will have its own isolated data and devices. A user authorized for **House A** must not be able to access the cameras, lights, residents, or security events belonging to **House B**.



CONCLUSION

The study of the existing residential management approach highlights several limitations related to security, lighting control, emergency response, and centralized management. The proposed **Intelligent House System** addresses these challenges by combining IoT, Artificial Intelligence, facial recognition, smart lighting, and a centralized mobile application.

The system aims to transform traditional residential management from a **manual and reactive approach into an intelligent and proactive environment**, improving security, energy efficiency, convenience, and responsiveness.



CHAPTER III : SPECIFICATION BOOK



Design and Implementation of an Intelligent House System Based on Light Intensity Control and Security Monitoring.



Content :

INTRODUCTION

I. CONTEXT AND JUSTIFICATION OF PROJECT

II. PROJECT OBJECTIVES

III. EXPRESSION OF NEEDS

IV. PROJECT PLANNING

V. ESTIMATED COST OF PROJECT

VI. PROJECT CONSTRAINTS

VII. DELIVERABLES

CONCLUSION



INTRODUCTION

The specification book is a crucial part in software engineering process. For this reason, care must be taken when elaborating this document in order for the solutions to clearly satisfy the client's needs. This document permits us specify the expectations of the client(s), there by establishing a certain agreement between the client(s) and the person(s) to realize or develop the project. This specification book is to avoid inadequate production result

I. CONTEXT AND JUSTIFICATION

1. CONTEXT

The residential environment faces significant challenges in ensuring effective home security, controlling lighting efficiently, and monitoring connected devices. Traditional homes often rely on manual lighting control and human observation for security, which can result in unnecessary energy consumption, delayed detection of suspicious activities, and limited control over household devices. In addition, conventional security systems may detect movement without being able to intelligently identify or analyse the nature of the detected event.

To address these issues, this study proposes the conception and implementation of an **Intelligent House System for Smart Lighting Control and Security Monitoring**. The proposed system, named **Vigilis**, will centralize the management of household devices, automate lighting control, monitor security devices, and integrate artificial intelligence for facial recognition and threat analysis. The system will use sensors and cameras to collect information from the environment and provide appropriate responses based on the analysis of detected events.

The system will also allow authorized users, particularly the homeowner and residents, to manage accounts, manage residents, activate security devices, monitor the house, control lighting, and receive security alerts. By integrating IoT devices, artificial intelligence, and a centralized application, Vigilis will improve home security, simplify household management, and provide a more responsive and intelligent residential environment.

2. JUSTIFICATION

Implementing an intelligent house system such as **Vigilis** is essential to improve security, convenience, and efficiency in residential environments. The proposed system will provide a centralized platform for managing household users, security devices, cameras, sensors, and lighting while enabling intelligent analysis of security-related events.

The system is justified by the need to reduce dependence on manual monitoring and provide faster and more appropriate responses to detected situations. By combining IoT devices with artificial intelligence, the system can collect environmental information, analyse detected events, identify authorized or unauthorized persons through facial recognition, and determine whether a situation may represent a potential security threat.

Key benefits include:

- **Centralized house management:** authorized users can manage residents, accounts, security devices, and lighting from a single application.
- **Improved security monitoring:** cameras and sensors can continuously monitor the residential environment and detect suspicious activities.
- **Facial recognition:** the system can assist in identifying or verifying persons detected within the monitored environment.
- **AI-based threat analysis:** detected events can be analysed to distinguish normal activities from potential security threats.

- **Smart lighting control:** lighting can be switched on or off and its brightness can be adjusted according to the user's needs or environmental conditions.
- **Real-time alerts:** users can receive notifications when relevant security events are detected.
- **Emergency response:** when a threat is sufficiently verified according to the system's configured rules, an emergency-response workflow can be initiated.
- **Remote accessibility:** authorized users can monitor and control the house through the application.
- **Improved energy efficiency:** automated lighting control can help reduce unnecessary use of electrical lighting.
- **Service payment management:** the homeowner can use the application to pay for Vigilis installation or maintenance services through an integrated payment service.

Overall, the proposed system seeks to combine **IoT, artificial intelligence, security monitoring, and smart lighting control** to create a centralized and intelligent solution capable of making the home safer, more efficient, and easier to manage.

II. PROJECT OBJECTIVES

i. GENERAL OBJECTIVE

The general objective of this study is to design and implement an **Intelligent House System (Vigilis)** that centralizes household security and lighting

management, enhances real-time security monitoring, and supports intelligent threat detection and response. Specifically, the system aims to automate lighting control, monitor security devices, provide AI-driven facial recognition and threat analysis, facilitate secure access for authorized residents, and enable appropriate emergency responses, ultimately improving the safety, efficiency, and convenience of the residential environment.

ii. SPECIFIC OBJECTIVES

To achieve the general objective, the following specific objectives are targeted:

- **Centralized House Management:** Develop an intuitive application for managing household users, residents, security devices, cameras, and lighting from a centralized platform.
- **Smart Lighting Control:** Implement automatic and manual lighting control, including switching lights ON/OFF and adjusting brightness according to environmental conditions or user requirements.
- **Security Monitoring:** Integrate security devices such as cameras and motion sensors to continuously monitor the residential environment and detect suspicious activities.
- **Facial Recognition and Threat Analysis:** Integrate AI-based facial recognition and threat analysis to identify or verify detected persons and analyse security events in order to identify potential threats.
- **Emergency Response:** Implement an emergency mechanism that allows authorized residents to trigger an emergency response and enables the system to initiate the configured response when a sufficiently verified threat is detected.

- **User and Resident Management:** Provide mechanisms for the homeowner to create, update, deactivate, and manage resident accounts and their access rights.
- **Real-Time Notifications:** Provide security alerts and notifications to authorized users when relevant events or potential threats are detected.
- **Data Security:** Ensure secure authentication, authorization, and protection of household, resident, device, and security information.
- **Service Payment:** Provide a payment functionality allowing the homeowner to pay for Vigilis installation or maintenance services through an integrated payment service.
- **House Isolation:** Ensure that users and devices are associated with the correct house, preventing unauthorized access or communication between devices belonging to different houses.

Project Characteristics

- **Project Name:** Vigilis – Intelligent House System
- **Project Target:** Smart lighting management, security monitoring, intelligent threat analysis, and emergency response.
- **Technical Specification:** Flutter/Dart frontend, Node.js/Express.js backend, Sequelize ORM, MySQL database through XAMPP, ESP32-based IoT devices, cameras, sensors, and AI-based analysis.

III. EXPRESSION OF NEEDS

i. FUNCTIONAL NEEDS

These are the requirements that the system must meet in order to fulfill its purpose, typically expressed in terms of the system's inputs, outputs, and behaviours. They are as follows:

a. Homeowner / Administrator

The homeowner is the main administrator of the house and is responsible for managing the residential environment.

Manage Accounts

- Create resident accounts
- Update resident account information
- Deactivate or delete resident accounts
- Manage user access rights

Manage Residents

- Add residents
- Update resident information
- Delete or deactivate residents
- Assign appropriate access permissions

Manage Security

- Activate security devices
- Deactivate security devices
- View security status
- Monitor security events

- Manage security settings

Manage Lighting

- Turn lights ON
- Turn lights OFF
- Adjust brightness
- Monitor lighting status
- Enable automatic lighting control

Manage Devices

- Add connected devices
- View connected devices
- Update device information
- Remove devices
- Monitor device status

Manage Payments

- View available installation or maintenance services
- Initiate a payment
- Select a payment method
- View payment status
- View transaction history

b. Resident

Authorized residents can interact with the house according to the permissions granted by the homeowner.

Control Lighting

- Turn lights ON/OFF

- Adjust brightness

Monitor Security

- View security status
- Receive security notifications
- View relevant security events

Emergency Response

- Trigger the emergency button when immediate assistance is required

c. Security / AI System

Monitor Security Events

- Receive information from security sensors
- Detect motion
- Capture images or video through cameras
- Analyse detected persons

Facial Recognition

- Detect a person's face
- Compare the detected face with authorized resident information
- Identify or classify the person as authorized or unknown

Threat Analysis

- Analyse detected security events
- Combine available evidence from sensors and cameras
- Determine whether an event represents a potential threat
- Generate a security alert when necessary

Emergency Response

- Trigger the configured emergency-response workflow when a sufficiently verified threat is detected
- Notify the appropriate security or emergency service when the required integration is available

IV. NON-FUNCTIONAL NEEDS

It specifies the quality attributes of the software system. They evaluate the system based on its performance, usability, reliability, scalability, security, maintainability, and other standards that are critical to its success.

i. Usability

User Interface (UI) Design

- The system should provide a user-friendly, intuitive, and easy-to-navigate interface.
- The main functions such as lighting control, security monitoring, resident management, and emergency response should be easily accessible.
- The interface should allow authorized users to understand the current state of the house quickly.

Purpose: Enhances user satisfaction and facilitates efficient interaction with the system.

ii. Performance

Response Time

- The system should respond to user actions, such as switching lights, adjusting brightness, activating security devices, and managing residents, within acceptable time limits.

- Security events should be processed and communicated to authorized users with minimal delay.

Purpose: Ensures efficient operation and reduces delays during security-related events.

iii. Scalability

- The system should be capable of supporting an increasing number of residents, connected devices, sensors, and security events without significant degradation in performance.
- The architecture should allow the system to be deployed in multiple houses while maintaining isolation between households.

Purpose: Allows the system to grow and support additional residential installations.

iv. Reliability

Availability

- The system should remain available to authorized users with minimal downtime.
- Critical security-monitoring functionalities should remain operational whenever the security system is activated.

Purpose: Ensures continuous access to house-management and security information.

Fault Tolerance

- The system should handle communication failures between the application, backend, database, and IoT devices appropriately.
- Device failures should not cause the entire system to become unavailable.

v. Security

Data Protection

- User, resident, device, security-event, and payment information should be securely stored and protected against unauthorized access.
- Sensitive information should not be exposed to unauthorized users.

Purpose: Protects household information and maintains user trust.

Authentication and Authorization

- The system should implement secure authentication mechanisms.
- Role-based authorization should ensure that users can only access functions and devices for which they have permission.
- A resident should not be able to access administrative functions unless explicitly authorized.

Purpose: Prevents unauthorized access and maintains data confidentiality.

House Isolation

- Each house should have a unique identifier.
- Users and devices should be associated with their respective house.
- Requests should be validated to ensure that a user cannot access or control devices belonging to another house.

Purpose: Prevents cross-house communication and unauthorized device control.

vi. Maintainability

Code Maintainability

- The system should use a modular architecture to facilitate maintenance and future modifications.
- The Flutter frontend, Express.js backend, database models, IoT communication, and AI components should be separated into appropriate modules.
- The code should be documented and structured to facilitate debugging and future development.

Purpose: Makes the system easier to update, debug, and extend.

V. PROJECT ESTIMATED COST

i. SOFTWARE RESOURCES

For the realization of the **Intelligent House System**, the following software resources will be required.

Table 5: Software estimated cost

Software Resource	Description	Quantity	Price (FCFA)/day	
Microsoft windows10pro (64 bits)	Operating System	01	57247	5

Microsoft office 365	It is a productivity suit that includes excel, word, outlook etc. to create documents, spreadsheets, presentations and emails	01	37247	3
Visual Paradigm	Modeling tool	01	Freemium	1
Icogram	Card-graphic tool	01	Freemium	1
Visual Studio	Front-end development environment	01	Freeware	1
Gantt Chart	Planification tool	01		
Node js	Back End development environment	01	Freeware	1

ii. HARDWARE RESOURCES

The hardware resources include the equipment required for developing, testing, and demonstrating the Intelligent House System.

Table 6: Hardware estimated cost

Hardware resource	Quantity	Price (FCFA)/day	Total price (FCFA)
laptop 8G of RAM, SSD, corei5, processor 1.10GHz	02	591394	1182788
Modem (Camtel)	01	50000	50000
Hard Disk Drive (500Gb)	01	50000	50000
USB Key(16Gb)	01	10000	10000
Internet Connection(MTN)	/	126500	126500
Total Cost Of Software Resource (FCFA)			1419288

iii. HUMAN RESOURCES

The project requires different roles for analysis, design, development, testing, and supervision.

Table 7: Human resources estimated cost

Human Resource	Quantity	Price (FCFA/day)	Duration	Total Price
Head of Project	01	30000	31 days	930000
Analyst	01	25000	21 days	525000
Developer	01	20000	30 days	600000
Tester	01	10000	10 days	100000
TOTAL				2155000

VI. TOTAL ESTIMATED COST FOR THE PROJECT

Table 8: Total cost

SOFTWARE RESOURCE	94495 FCFA
HARDWARE RESOURCE	1419288 FCFA

HUMAN RESOURCE	2155000 FCFA
Unexpected charges (10%) (Total cost) * 10%	1200000 FCFA
TOTAL COST	4868782 FCFA

TOTAL : 4,868,782 FCFA

VII. PROJECT PLANNING

Table 9: Project planning

Phase	Objective	Outing	Duration
INSERTION	Collection of information on the enterprise	Insertion report	14 days
EXISTING SYSTEM	Description of the existing system	Existing system	6 days

SPECIFICATION	Define the project scope, goals and functional requirements	Specification book	11 days
ANALYSIS	➤ Study of the existing system ➤ Use case and textual description ➤ modelling	Analysis document	15 days
CONCEPTION	➤ outline system architecture ➤ design the database schema ➤ design the user interface	Conception document	10 days
REALIZATION	➤ set up development environment ➤ implement system and database ➤	Realization document and source code	21 days
TESTING	➤ perform unit testing ➤ test for performance	User testing	5 days

USER GUIDE	User guide of the solution	User manual and guides	5 days
PRESENTATION	Realization of the PowerPoint and presentation of the work realized	Power point	5 days

VIII. PARTICIPANTS

The table below presents the different persons involved in the accomplishment of the project.

Table 10: Participants

Names	Functions	Role and Tasks
Mr TCHOUA NANA	Entrepreneur, Lecturer at AICS-Cameroon	CEO of INTERLINK
Mr YOGO JEAN ARIEL LEPRECIEUX	second year Student at AICS-Cameroon	Project head, analyst, conception and coding of project, testing
Mr BANGOU RUPHIN	third year Student at AICS-Cameroon	Professional in presentation

A. Gantt Diagram

The Gantt project software stands as a highly effective instrument for illustrating the diverse tasks within a project, their respective dates and the precise sequence in which they are executed.

Figure 3: Gantt chart

VII. PROJECT CONSTRAINTS

To implement the **Intelligent House System**, three main constraints must be respected:

- **Budget Constraint:** The fixed budget allocated for the realization of the project must be respected. Particular attention must be given to the cost of IoT components, development resources, internet connectivity, and any external services required by the system.
- **Deadline Constraint:** The development of the system must respect the assigned deadline. The project should be completed within the planned development period, including analysis, design, implementation, integration of IoT and AI components, testing, documentation, and final demonstration.
- **Quality Constraint:** Despite the budget and time limitations, the system must provide a reliable, secure, flexible, maintainable, and reusable solution. The application should be capable of integrating additional smart-home devices and supporting future improvements without requiring major modifications to the entire system.

VIII. DELIVERABLES

This section presents the different elements that will be delivered at the end of the project. They include:

- **The Vigilis application:** A Flutter-based application allowing authorized users to manage residents, control lighting, monitor security devices, receive alerts, and interact with the intelligent house system.
- **Backend and database:** The Node.js/Express.js backend with Sequelize and MySQL database used to manage users, residents, houses, devices, security events, and other system data.
- **IoT components and source code:** The ESP32-based components, sensors, camera modules, and associated programs required to demonstrate the intelligent house functionalities.
- **Installation guide:** Documentation describing how to install, configure, and connect the application, backend, database, and IoT devices.
- **User manual:** Documentation explaining how the homeowner and authorized residents can use the different functionalities of the system.
- **UML documentation:** The different diagrams produced during the analysis and design phases, including use-case, activity, sequence, communication, class, and state-machine diagrams.
- **Technical documentation:** Documentation describing the architecture, technologies, database structure, APIs, and major technical components of the system.
- **PowerPoint presentation:** A presentation summarizing the problem, objectives, analysis, conception, implementation, results, and demonstration of the Vigilis system.

CONCLUSION

To conclude, the specification book has enabled us to define the general and specific objectives of the **Vigilis Intelligent House System**, identify its functional and non-functional requirements, establish the project constraints, determine the required software, hardware, and human resources, and define the project planning and expected deliverables.

The proposed system aims to provide a centralized and intelligent solution for **home security, resident management, smart lighting, device monitoring, facial recognition, and AI-assisted threat detection**. The requirements and constraints established in this document provide a solid foundation for the next stage of the project.



CHAPTER IV : ANALYSIS PHASE



INTRODUCTION

1. METHODOLOGY

2. Modeling

CONCLUSION



INTRODUCTION

Analysis is the decomposition of a problem into its various elements for computer processing. Therefore, for the successful completion of a project, it is very important to carry out a good analysis in order to achieve the objectives specified in the specification book. The realization of a good project is mainly based on its analysis. Thus, a poorly analysed project cannot give good results.

1. METHODOLOGY

Analysis is the fundamental step in the conception of a software. It is the base to the realization of every information system. An information system is an organization of resources (human, software and hardware), data and feedbacks to attain a specific objective. Several methods and languages have been put in place to facilitate the analysis and conceptions of information systems which we principally have UML and MERISE. MERISE (Méthode d'Etude et de Realization Informatique pour les Systèmes d'Entreprise) it is a method of analysis, conception and realization of information systems mostly used in French Speaking Companies or Enterprises. It is based on the separation of data and processing them using several conceptual and physical models. Its principal objective is to conceive an information system. Merise proposes to consider the real system from two points of view I.e.

- ✓ The Static View (data);
- ✓ The Dynamic View (treatments).

It is a systematic and conceptual method of analysis.

UML (Unified Modeling Language) is a modeling language that gives the standardize visualized view of an information system. It uses diagrams to represent each aspect of a system in an object-oriented way.

UML proposes an approach different from MERISE as its readability and reusability makes it an ideal choice for programmers.

UML is currently at its 2.5 version and presents a number of advantages which includes:

- ✓ It is a formal and standardize language.
- ✓ It provides visual representation of the structure and behavior of the system which can help to communicate and understand the design of the system.

- ✓ It is flexible and customizable to suit different domain and technologies.
- ✓ It is readable and re-useable i.e. you can easily see the relationships and interactions between classes and objects in UM
- ✓ It is a planning tool that can be used to help design and document a system before the im plementation phase.

We can present the UML hierarchy as follows:

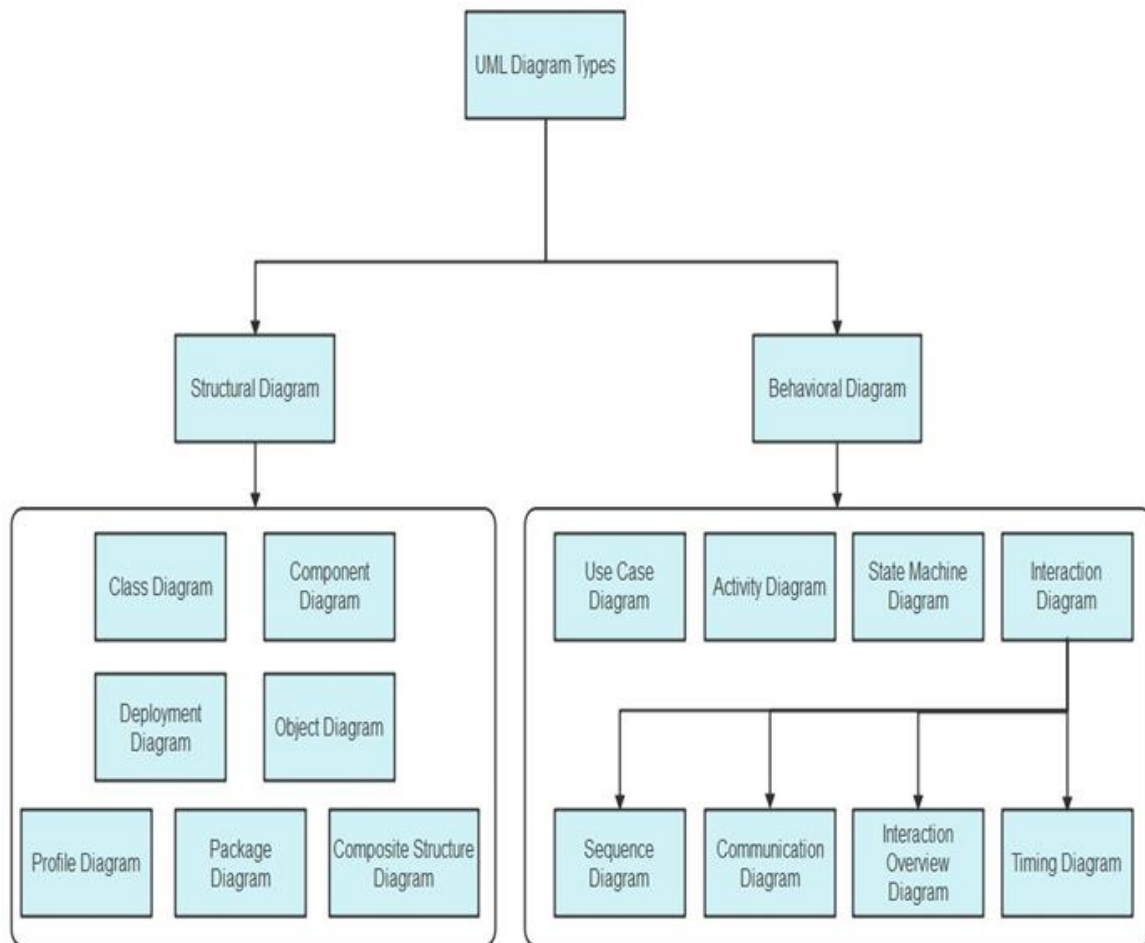


Figure 4: UML hierarchy

1.1 COMPARATIVE STUDIES BETWEEN MERISE AND UML

Table 11: Comparing between MERISE and UML

MERISE	UML
Méthode d'Etude et de Realization Informatique pour les Systèmes d'Entreprise	Unified Modeling Language
Merise is a method of analysis	UML is modeling language
Merise is a systematic method of analysis and design for information systems i.e. it uses a system approach	UML is however not a method but rather an object-oriented language, uses the 2TUP approach to model a system
It is based on procedural approach	Based on object approach

Merise proposes to consider the real system from two points of view:

- The Static view(data)
- The Dynamic view(treatments)

Which means in Merise we have a separate study of data and treatments

UML offers a different approach from that of

MERISE in that it combines data and process

1.2 PRESENTATION OF THE 2-TUP METHOD

2-TUP (Two Track Unified Process) is Y in shape and is a unified process which has as objective to bring a solution to constraints of functional changes imposed on information systems. It proposes a developments cycle which separates the functional aspects from the technical aspects. 2-TUP therefore distinguishes two main branches:

(The functional and technical branch) which their different results are being merged to realize our system.

Any unified process UP (Unifies Process) process meets the following characteristics:

It is iterative and incremental. Some of the advantages of the 2TUP method includes:

- ✓ It allows flexibility in the analysis of a project
- ✓ It is easy to design and understand
- ✓ It allows to have a preliminary view

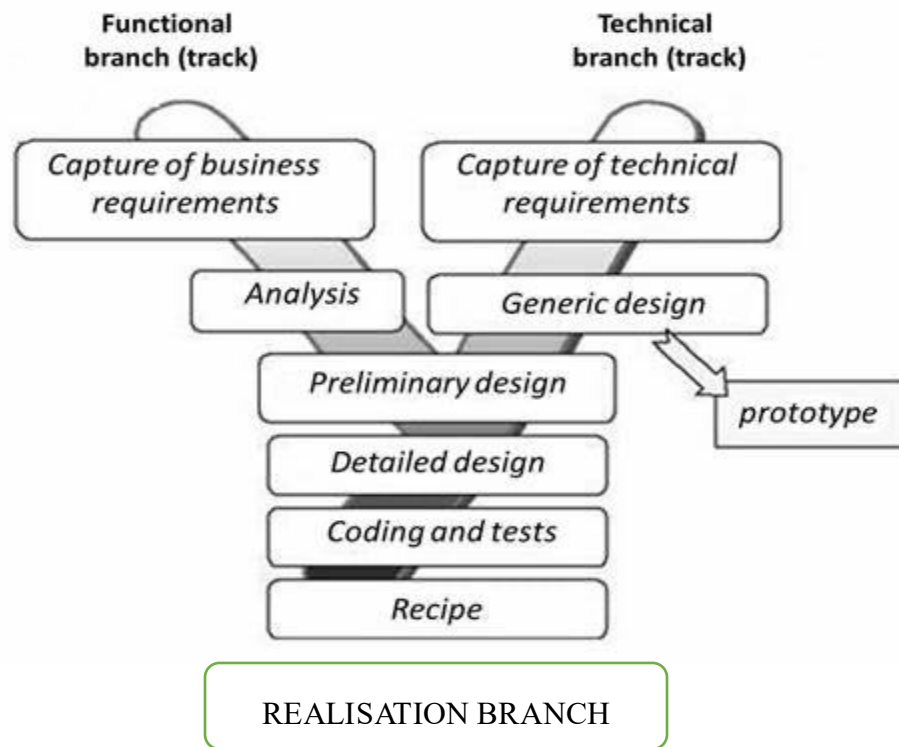


Figure 5: 2TUP representation

This process is articulated with these essential phases:

- i.) **The Functional Branch:** this branch captures business needs, resulting in a model focused on the end-user's requirements.
- ii.) **The Technical Branch:** this branch proposes a generic design from the technical needs, identifies the materials, tools and technologies to be used.
- iii.) **The Implementation:** The implementation phase consists of bringing the two branches together, providing the application design and finally the delivery of a solution adapted to the needs.

a) Functional Branch (left branch)

- ✓ **Capture of Business Needs:** this part minimizes the risk of producing an inadequate system with the needs of users and also verifies its consistency.
- ✓ **Analysis:** this is the study of the specifications to find out what the system will be made of in terms of the business rules. We have the following diagrams: Use Case, Activity Diagram, State Machine Diagram, Communication diagram

b) Technical Branch

- ✓ **Capture of Technical Needs:** this stage consists of the identification of tool, materials and technologies to use in developing the system. This technical architecture will be presented in this stage.
- ✓ **Generic Designs:** the technical architecture will be presented in this stage. We have the following diagram: mockUp7 Design.

c) Implementation Branch (middle branch):

- ✓ **Preliminary Branch:** this is the stage where the analysis model is integrated into the technical architecture. The goal here is to know what technical component will be used depending on the features of the analysis. We have the following diagrams: interaction overview, Component, Deployment, package diagram and Composite Diagram.
- ✓ **Detailed Design:** this is the detailed design of each feature of the system. We have the following diagrams: class, object, sequence and timing Diagrams.
- ✓ **Coding and Testing:** This is the programming phase of the designed features alongside testing of the coded features.
- ✓ **Recipe (Result):** this is the validation phase of the functions of the system developed.

1.3 JUSTIFICATION ON THE CHOICE OF THE ANALYSIS

Our choice was based on the modelling language UML associated to it a method of analysis 2TUP due to certain criterion which are as follows:

- ✓ UML is based on object-oriented approach;
- ✓ UML associate's data and treats them;
- ✓ It is a modelling language which focus mainly on users' needs and expectations;
- ✓ 2TUP is a process built from object-oriented approach and is built from UML;

2TUP facilitates the modelling of complex systems which can further be readjusted.

2. MODELING

2.1 USE CASE DIAGRAM

i.) Definition:

The use case diagram simply shows the functionalities of the system, their interdependency and how they can be linked with actors of the system. It also clearly defines the boundaries of the system.

ii.) Formalism:

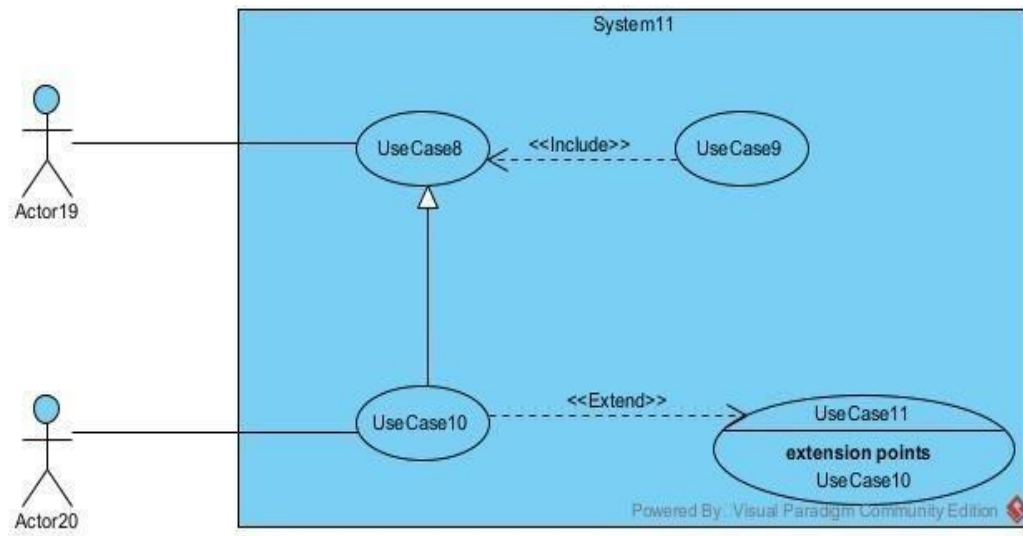
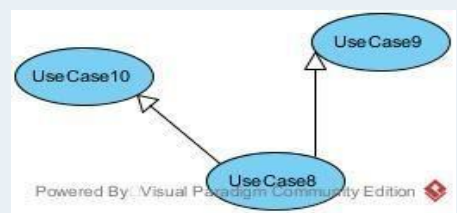
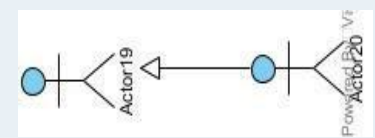
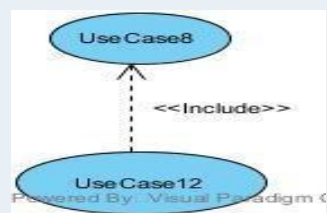


Figure 6: Formalism of use case

iii. Components of a Use Case Diagram

Table 12: Components of a use case diagram

Elements	Description and main properties	Representation
Actor	Represents the entity who directly interacts with the system. The actor is who performs the different task or actions of the system.	

Use Case	it represents a functionality of the system. It is an action that can be performed by an actor	
Generalization	It shows that a use case is a kind of another. This relationship permit the decomposition of a complex use case to simpler ones.	
Inheritance	this is the only possible relationship that can exist between actors whereby an actor can inherit from another	
Inclusion	It is denotes that an included action must be performed before another action can be done.	

iv) Graphical representation of use cases (General use case)

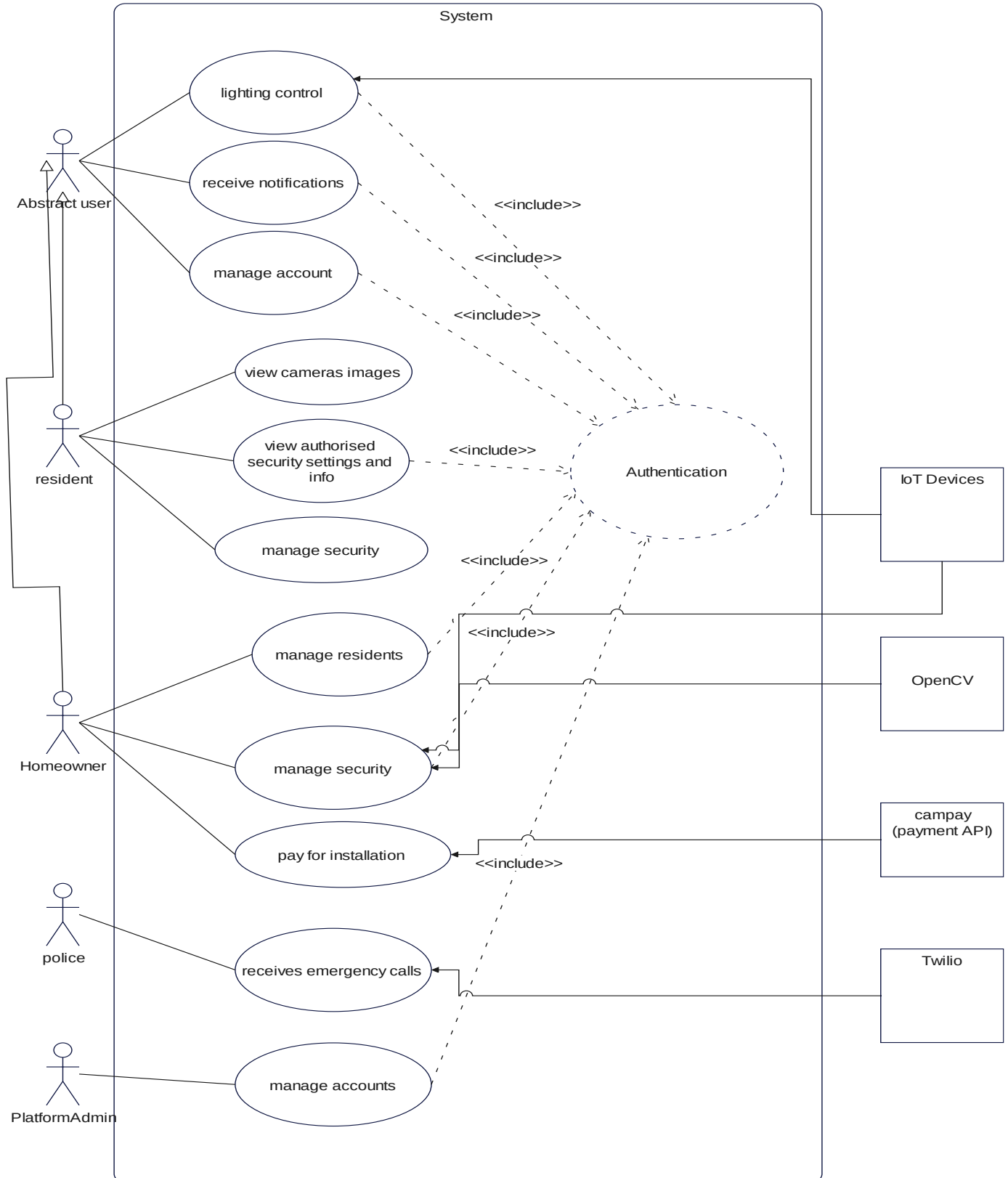


Figure 7: General use case diagram

Some specific use cases

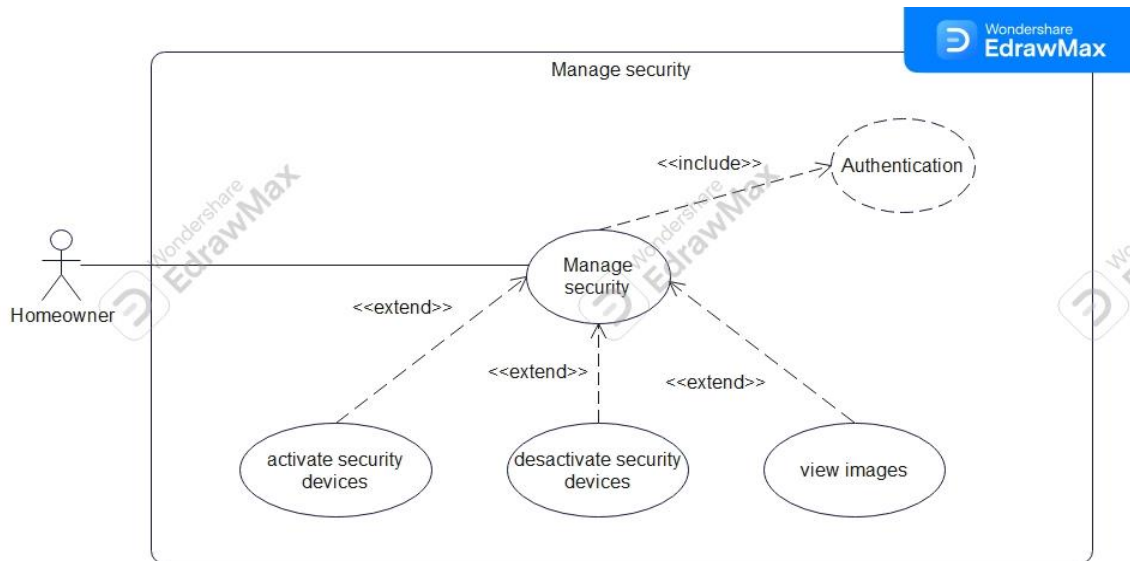


Figure 8:manage security specific use case

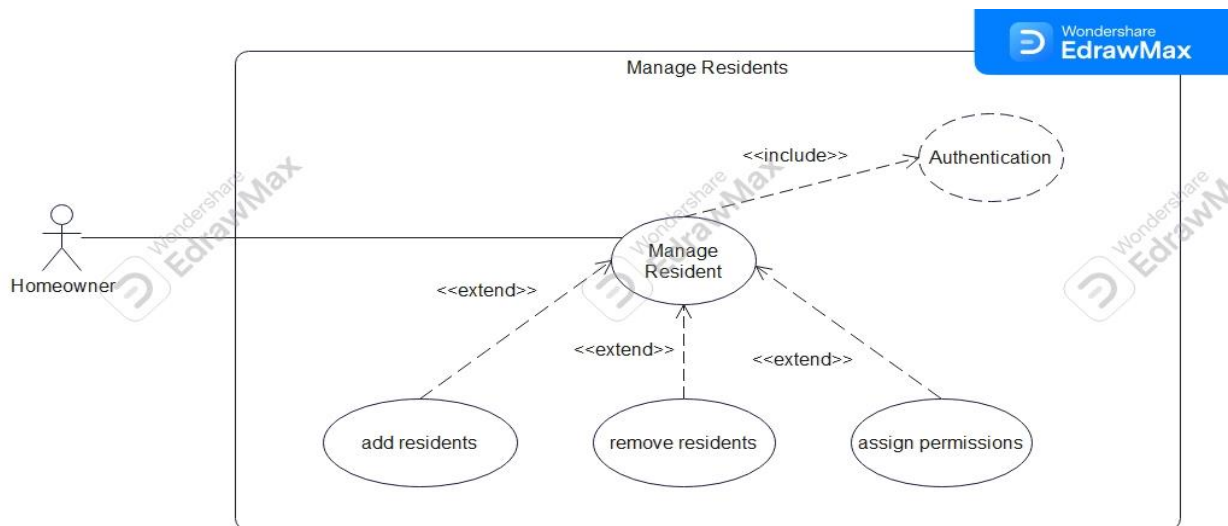


Figure 9: Manage residents specific use case

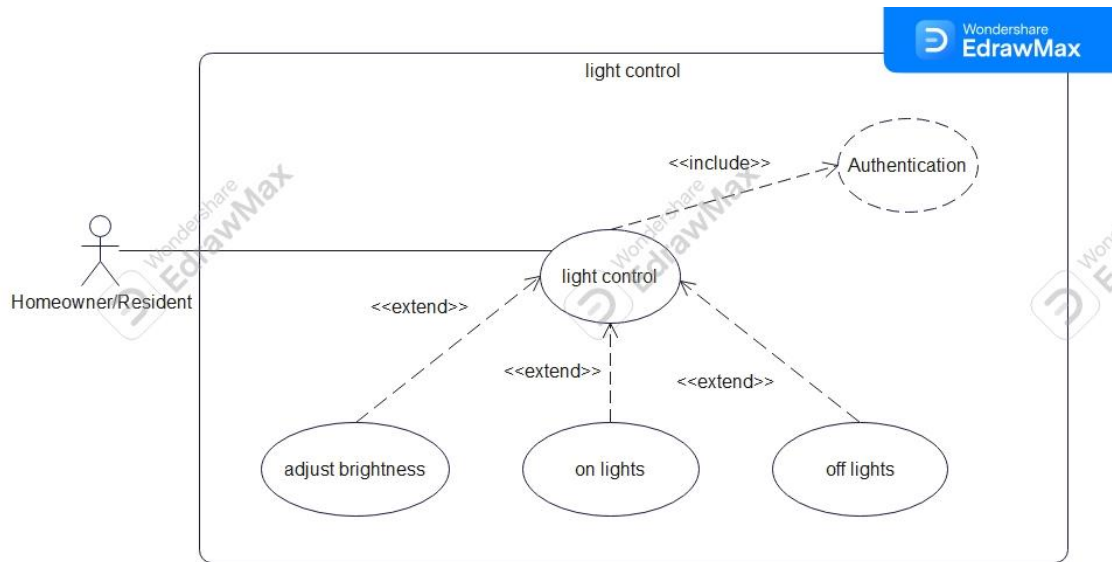


Figure 10: Light control specific use case

iv) TEXTUAL DESCRIPTION OF SOME USE CASES

Every use case is associated to a series of actions representing a given function. A use case is then an abstraction of a chain of executions. Implies an instance of a use case is known as a "scenario" which is a series of steps undertaken to execute a specific use case.

The Textual description is a simple text that describes how a use case can be performed in a more precise and understandable way following a chronological order. We will present in the following lines the textual description of some use cases.

1. Use case "manage residents"

Table 13:Description of manage residents use case

Element	Description
Use Case	Manage Residents
Actor	Homeowner

Objective	Allow the homeowner to manage residents who have access to the Intelligent House System.
Precondition	The homeowner must be authenticated and have administrator privileges for the house.
Trigger	Homeowner selects Manage Residents .
Nominal Scenario	1) Homeowner opens resident management. 2) System verifies authentication and authorization. 3) System displays the resident management interface. 4) Homeowner selects the required operation. 5) System executes the operation. 6) System displays the updated resident information.
Alternative Scenario	The homeowner can add residents , remove residents , or assign permissions .
Exception Scenario	If the operation cannot be completed, the system displays an appropriate error message.
Post-condition(s)	The resident information and permissions are updated.

2. Use case “manage security”

Table 14:manage security use case description

Element	Description
Use Case	Manage Security
Actor	Homeowner
Objective	Allow the homeowner to control and monitor the security devices installed in the house.
Precondition	The homeowner must be authenticated and security devices must be connected to the system.
Trigger	The homeowner selects Manage Security .

Nominal Scenario	1) Homeowner opens the security management interface. 2) System verifies authentication and permissions. 3) System displays the available security functions. 4) Homeowner selects the desired security operation. 5) System executes the selected operation. 6) System displays the updated security status.
Alternative Scenario	The homeowner may choose to activate security devices , deactivate security devices , or view camera images .
Exception Scenario	If a security device is unavailable or communication with the device fails, the system displays an error notification.
Post-condition(s)	The security devices are in the requested state or the requested camera images are displayed.

2.2 COMMUNICATION DIAGRAM

i. Definition

The communication diagram is used to model the specific interaction between the participants of the system. It describes both the static and dynamic behavior of the system. It shows the chronological succession of the actions performed by an actor in order to realize an operation.

ii. Formalism

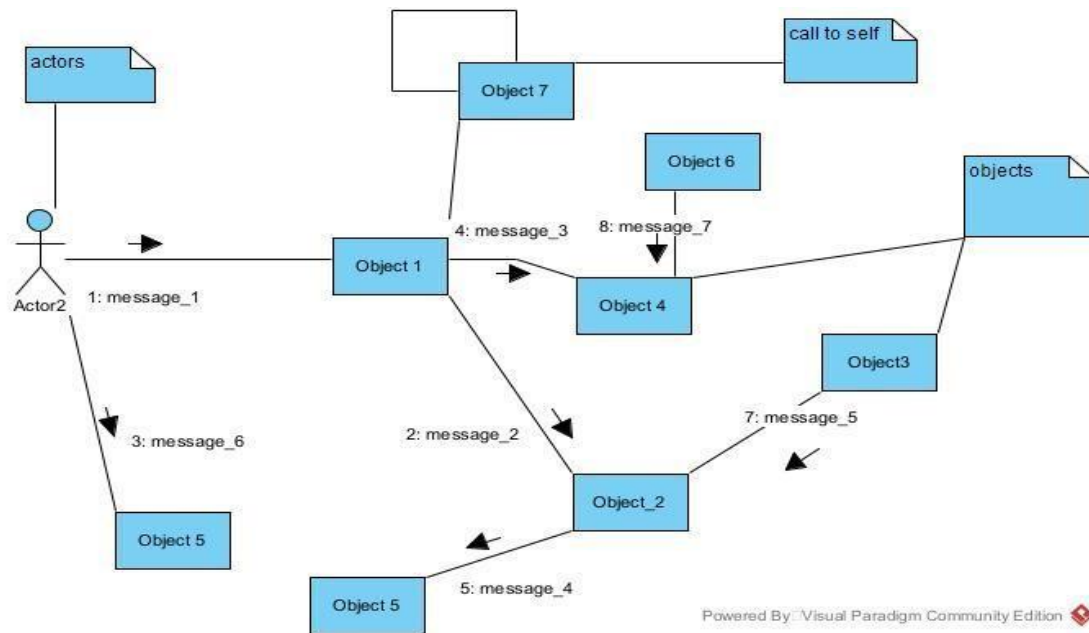
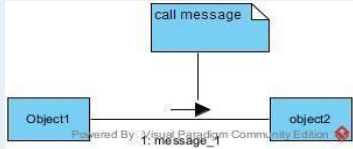
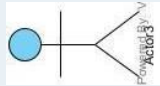
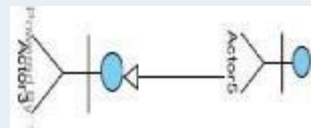
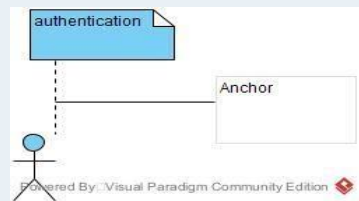


Figure 11: Formalism of communication diagram

iii. Components of a Communication Diagram

Table 15: Components of a communication diagram

Element	Description and properties	a Representation
Object	Represents a role or object instance that participated in the interaction model.	
Call Message	A message that represents invocation of	

	operation of the target object	
Actor	It models the type of role played by an entity that interacts with the subject	
Generalization	Existence between two related classes where one is a specialized form of the other	
Anchor	A connector that connects to a note shape for attaching additional description of the connected model element	

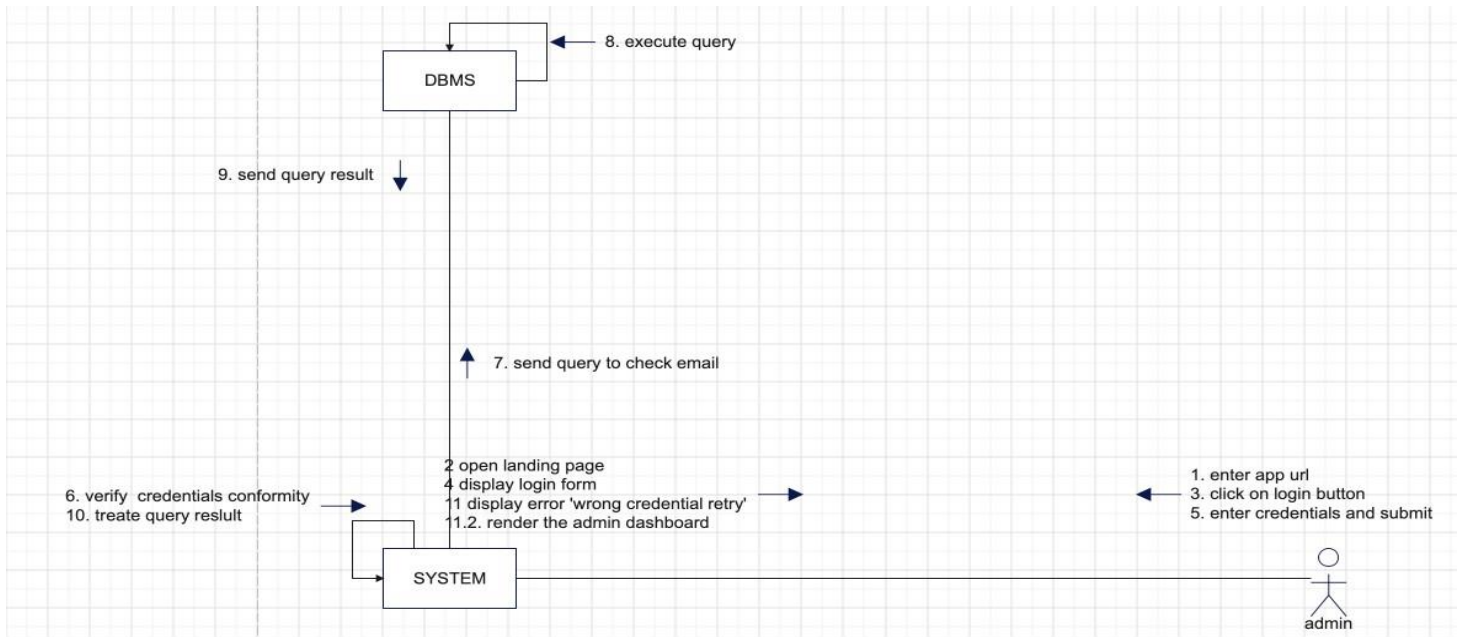


Figure 12: Communication diagram for Authentication

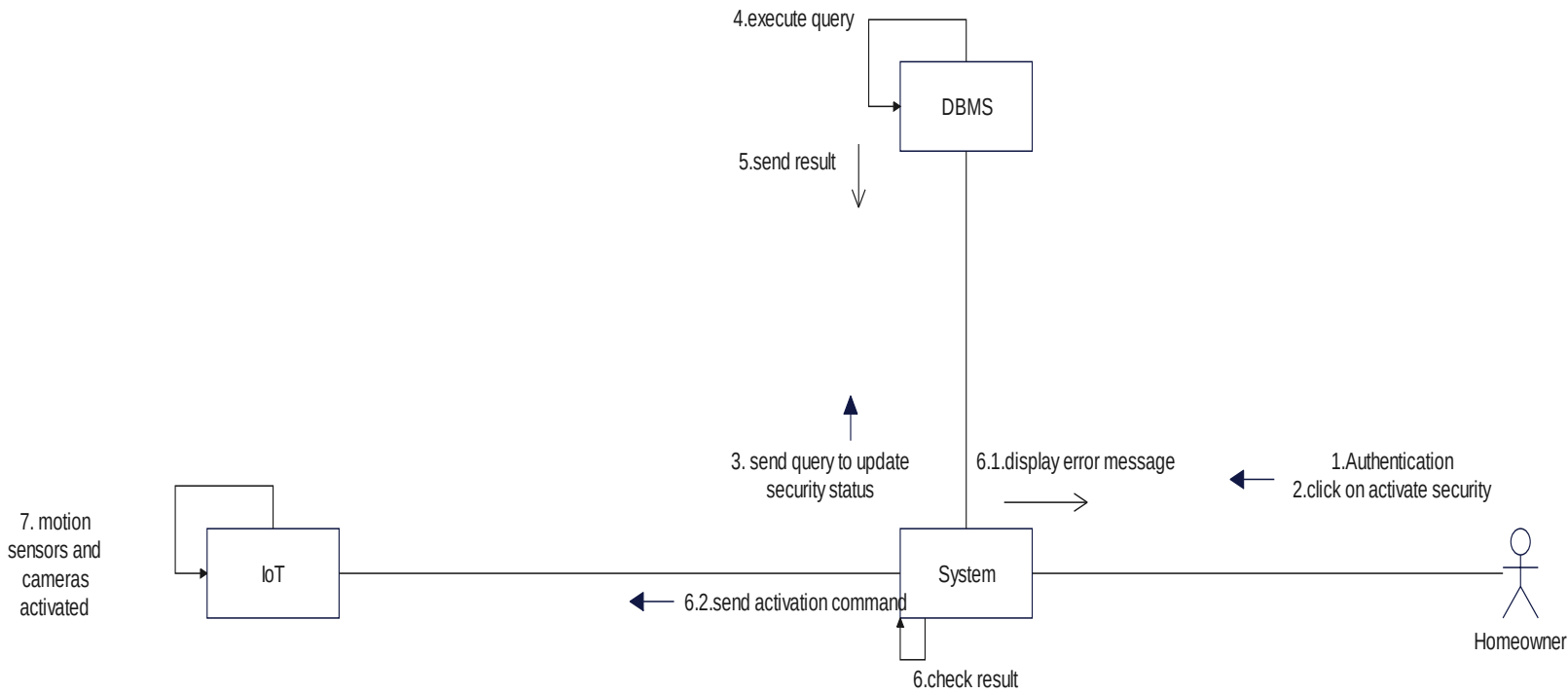


Figure 13: Activate security communication diagram

add resident
communication
diagram

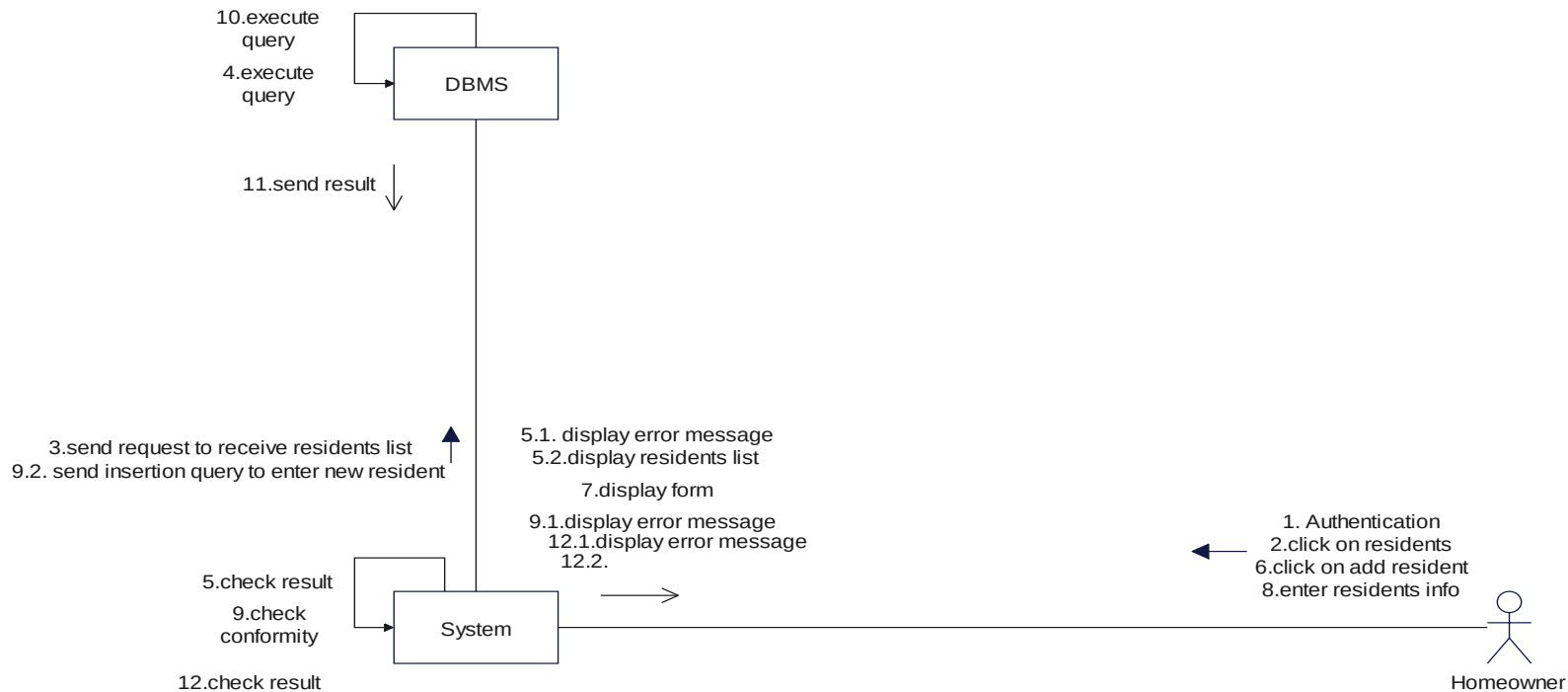


Figure 14: add resident communication diagrams

2.3. SEQUENCE DIAGRAM

i. Definition

A sequence diagram is a UML diagram that illustrates the sequence of messages between objects in an interaction. It consists of a group of objects that are presented by lifelines and the messages that they exchange over time during the interaction. These interactions are arranged from top to bottom following their order of occurrence.

ii. Formalism

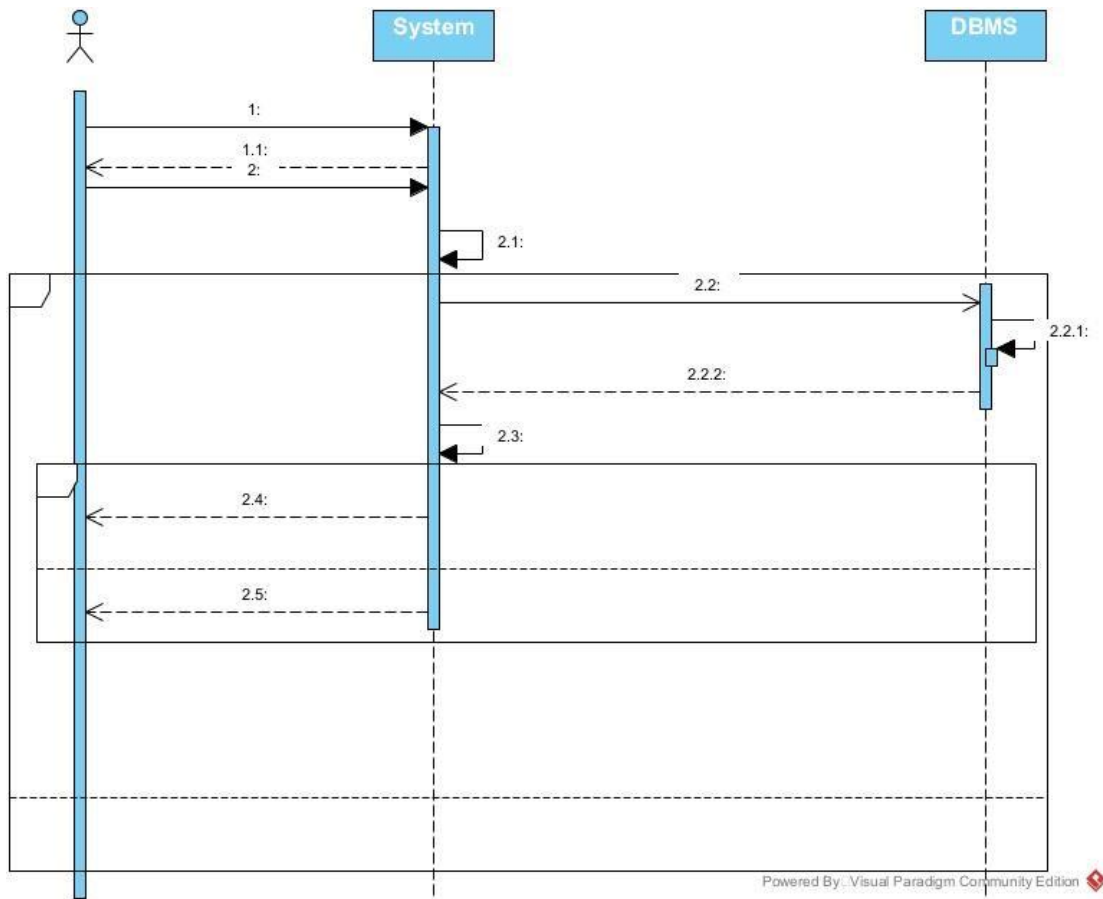
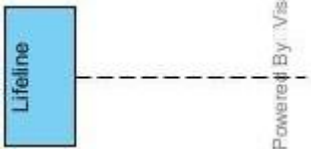
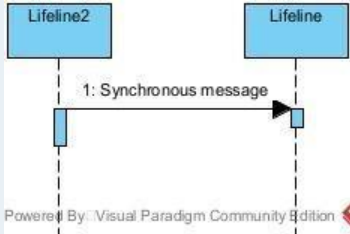
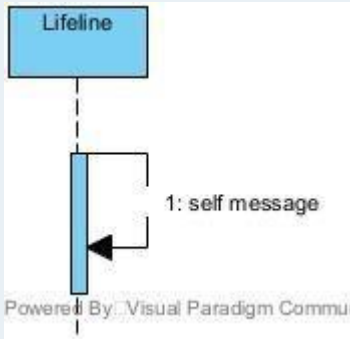
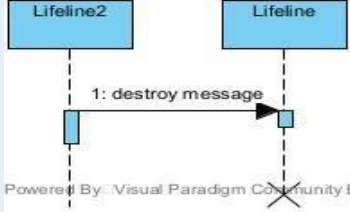
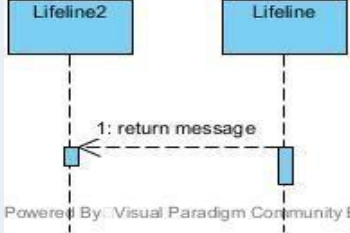


Figure 15: Formalism of sequence diagram

iii. Components of a Sequence Diagram

Table 16: Components of a sequence diagram

Element	Description and main properties	Representation
---------	---------------------------------	----------------

Lifeline	They represent rows or obj instances that participates in sequence being modeled.	
Synchronous Message	A message that represented an invocation of an operation of the target lifeline	
Self Message	Represents the execution operation call on the same objec	
Destroy Message	It destroys a participant in the course of a process	
Return Message	Represents a response to a message	

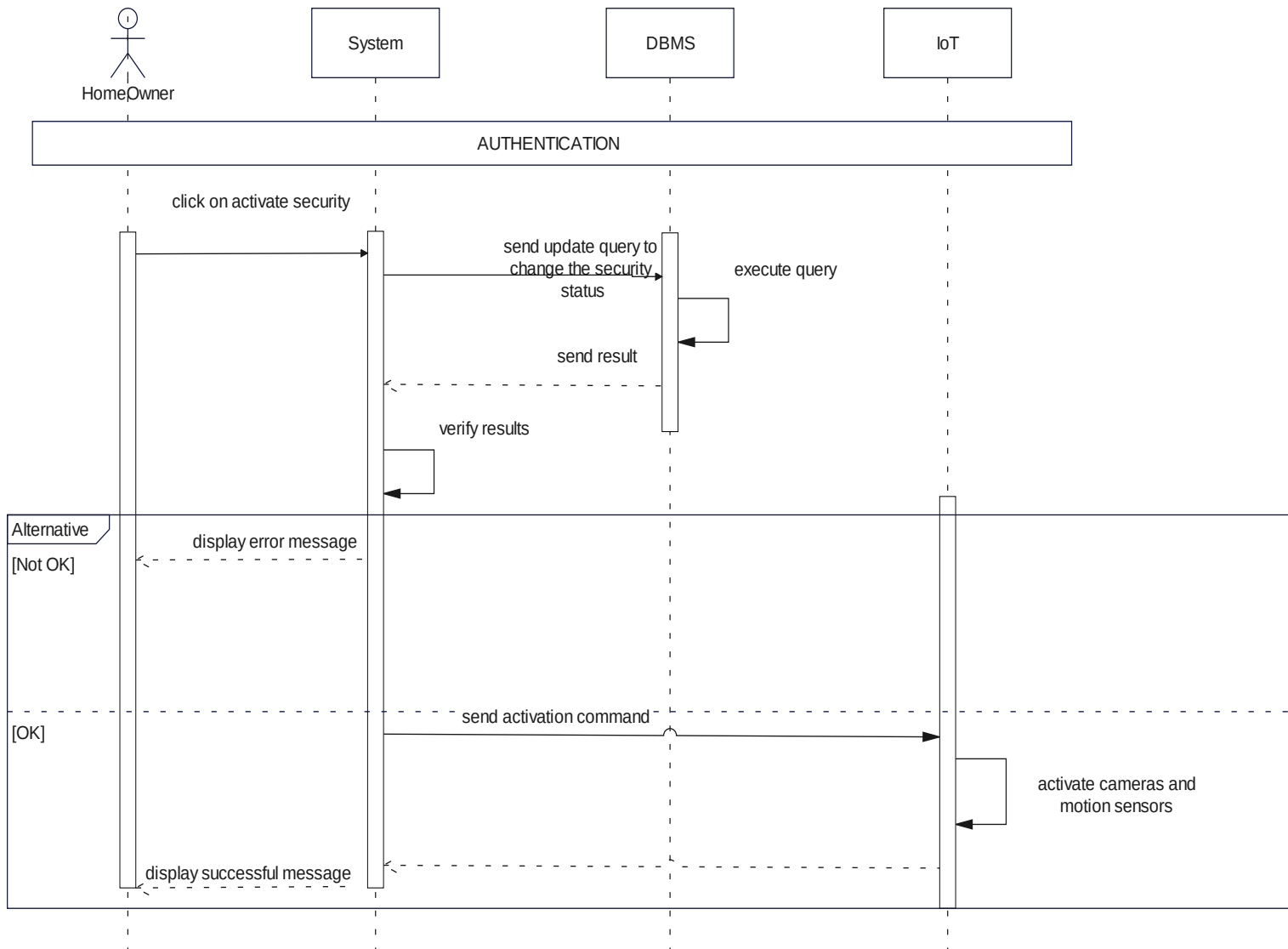


Figure 16: Activate security sequence diagram

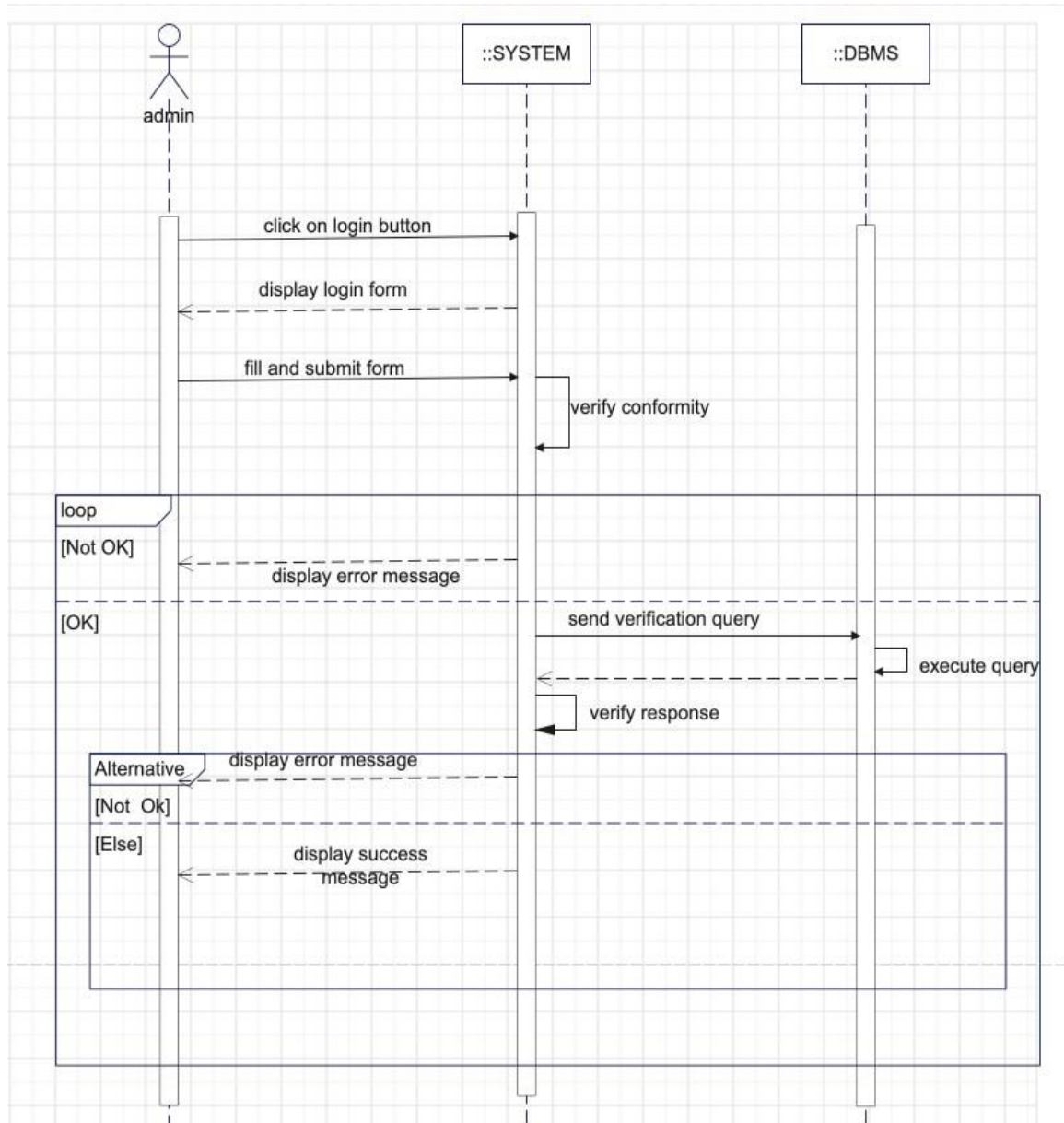


Figure 17: Sequence diagram for Authentication

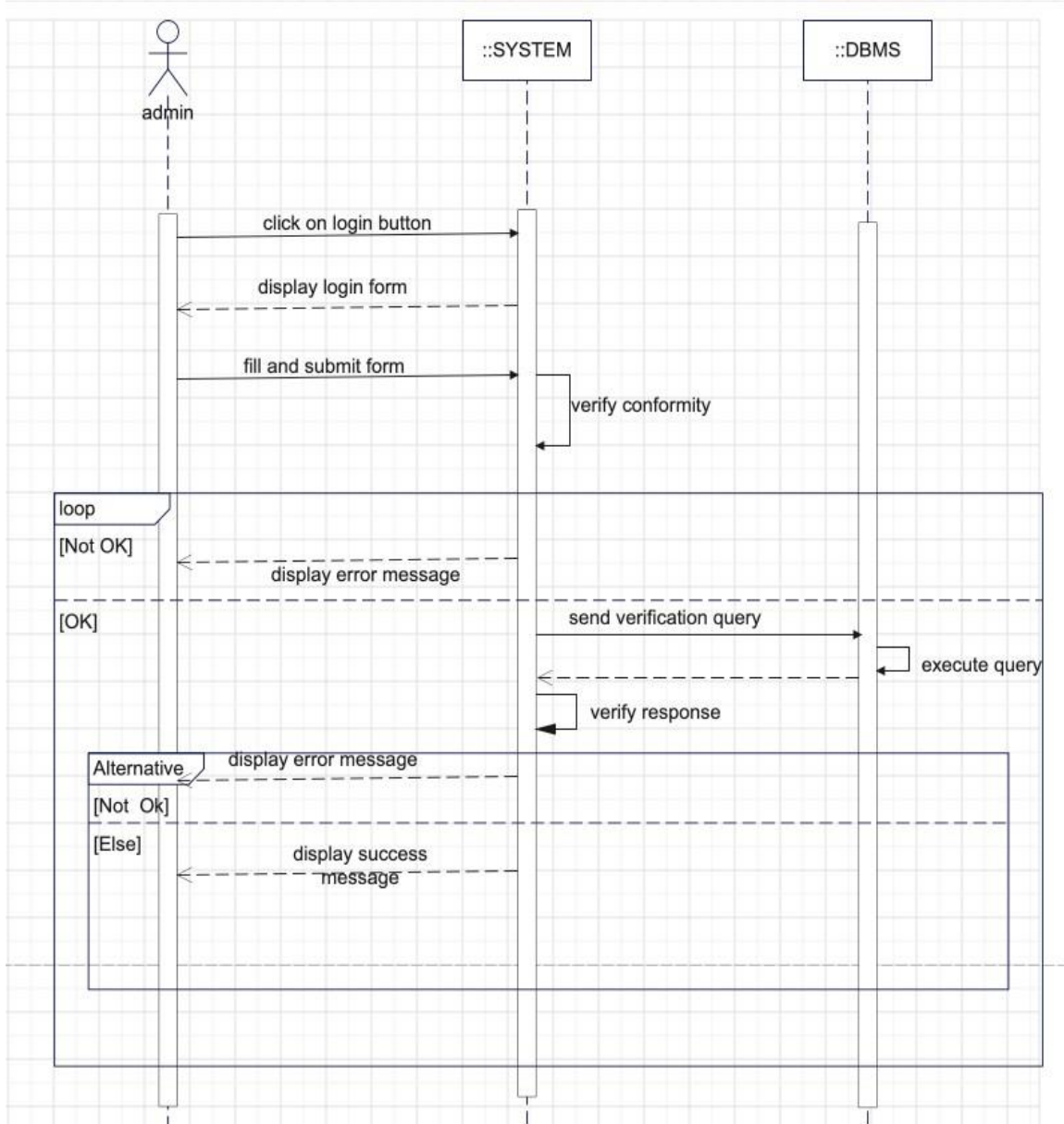


Figure 18: Sequence diagram for authentication

add resident
sequence
diagram

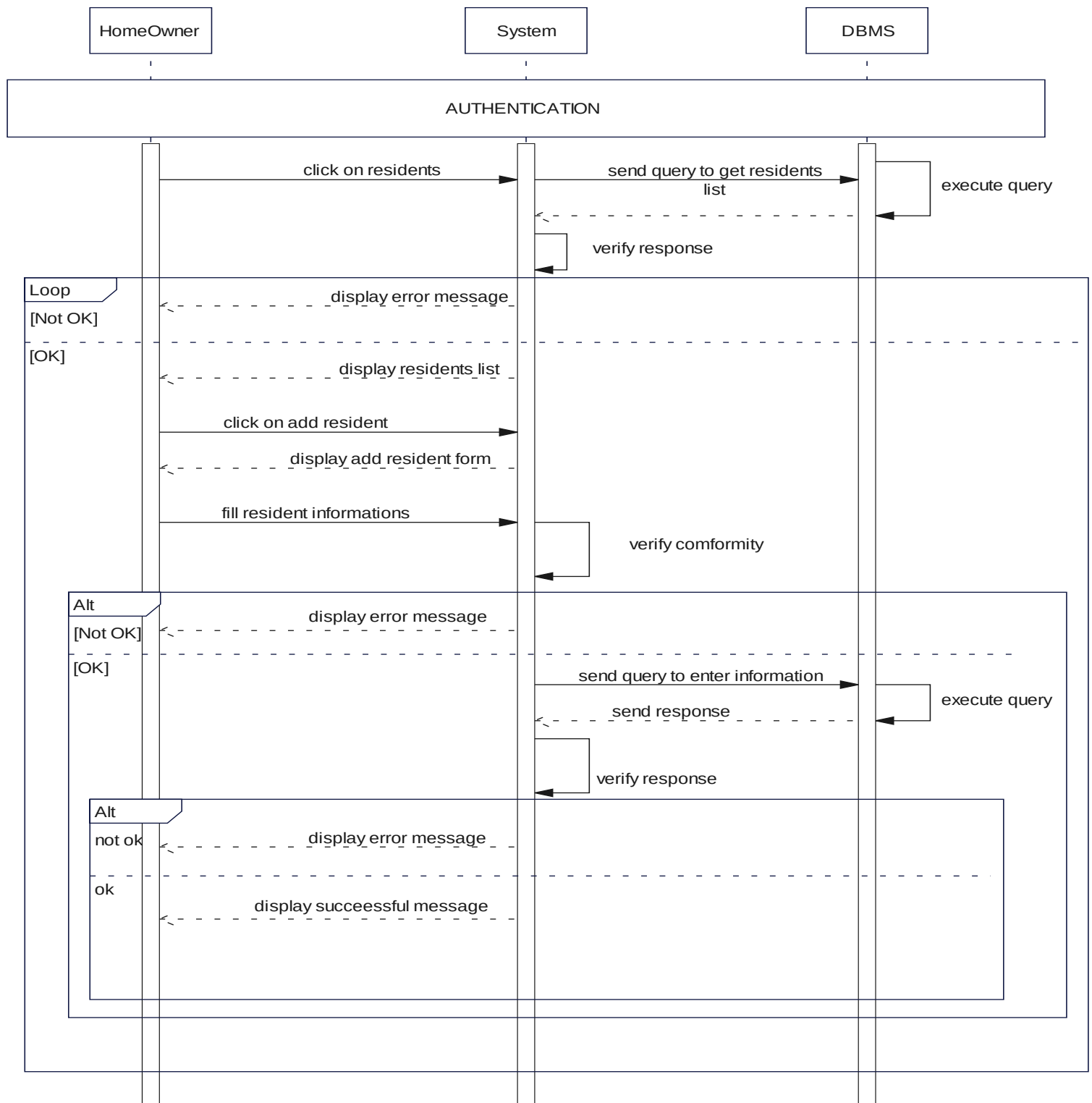


Figure 19: Add resident sequence diagram

2.4 ACTIVITY DIAGRAM

i. Definition

An activity diagram provides a view of the behaviour of a system by describing a sequence of actions in the process. The activity diagram makes it possible to emphasize the processing and to graphically formalize the sequence of actors carried out in a use case.

ii. Formalism

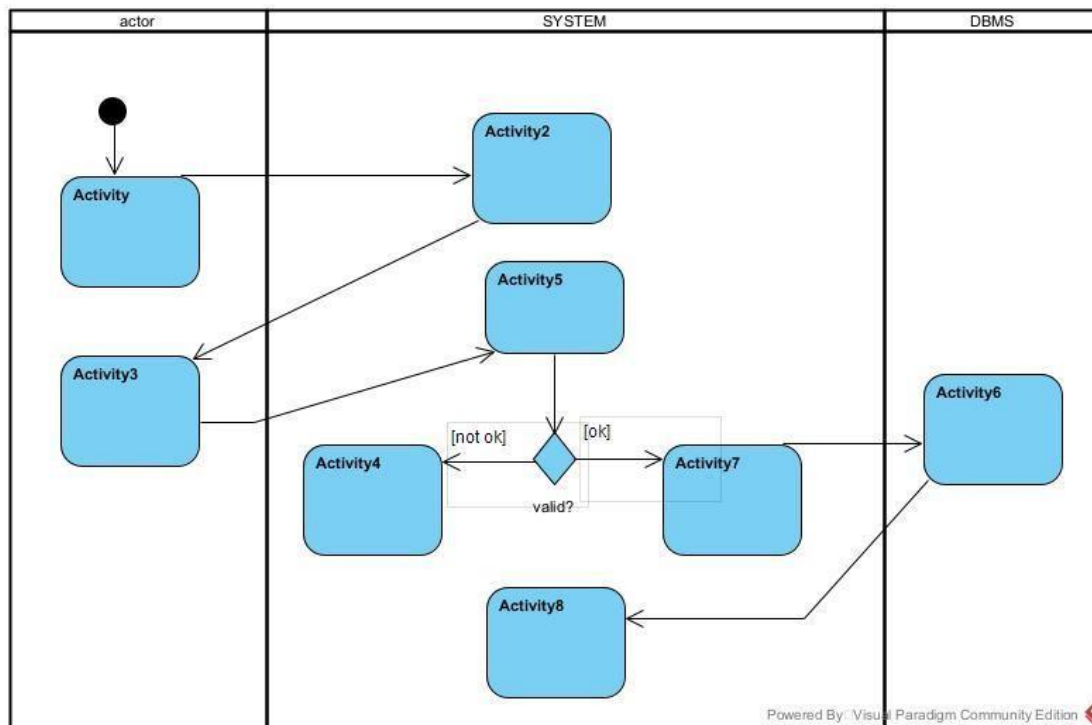
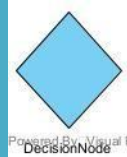
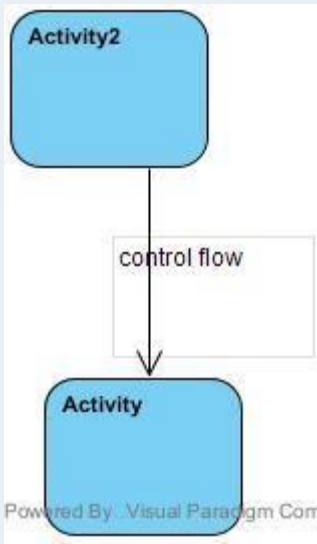


Figure 20: Formalism of activity diagram

iii. Components of an Activity Diagram

Table 17: Components of an activity diagram

Decision node	It is a decision node that takes a token (condition) from one edge and presents the different options or possibilities to cross possible outgoing edges. It is generally represented by a diamond	 DecisionNode
Initial Node	It is a control node at which flow starts when an activity is invoked	 Activity
Final Node	A control node that stops all flow in an activity. It is represented by a circle with a filled circle inside	
Control Flow	An edge that starts from an activity node after the previous one is finished	

Action

Represents a single step in activity

which cannot be further decomposed

h

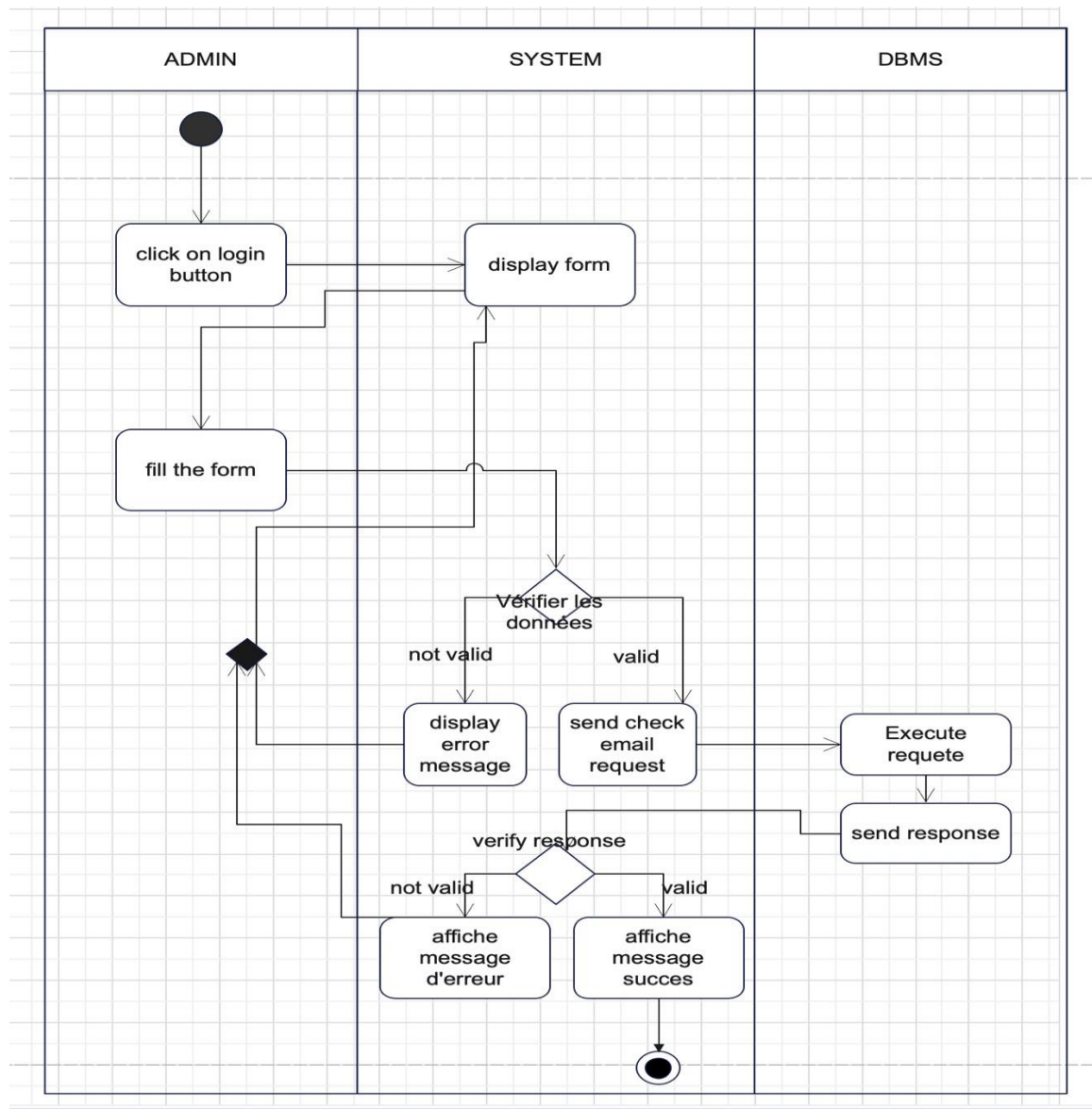


Figure 21: Activity diagram for authentication

Activity Diagram for Adding Residents

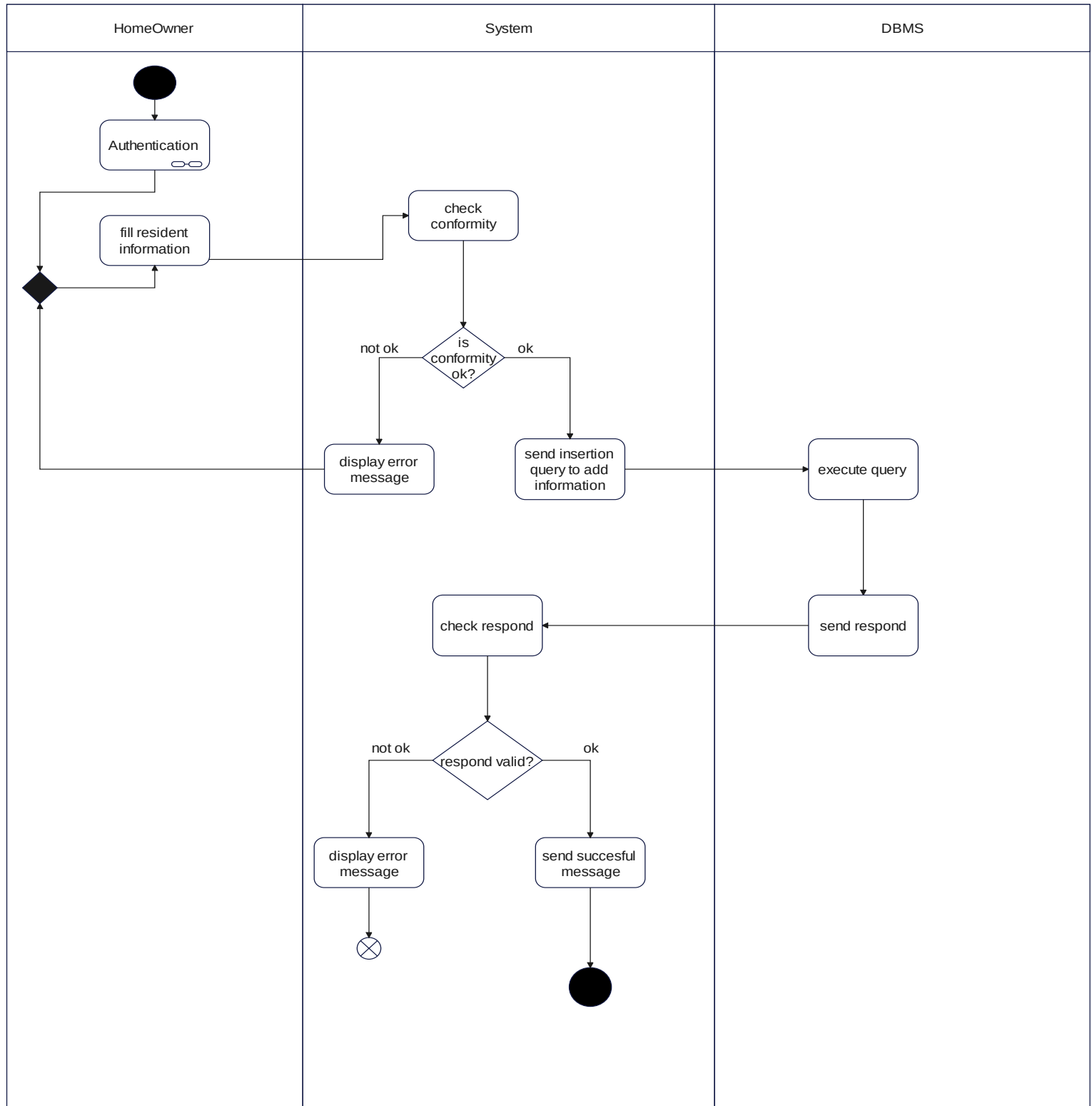


Figure 22:Activity diagram fo adding residents

Activity diagram for Activating Security

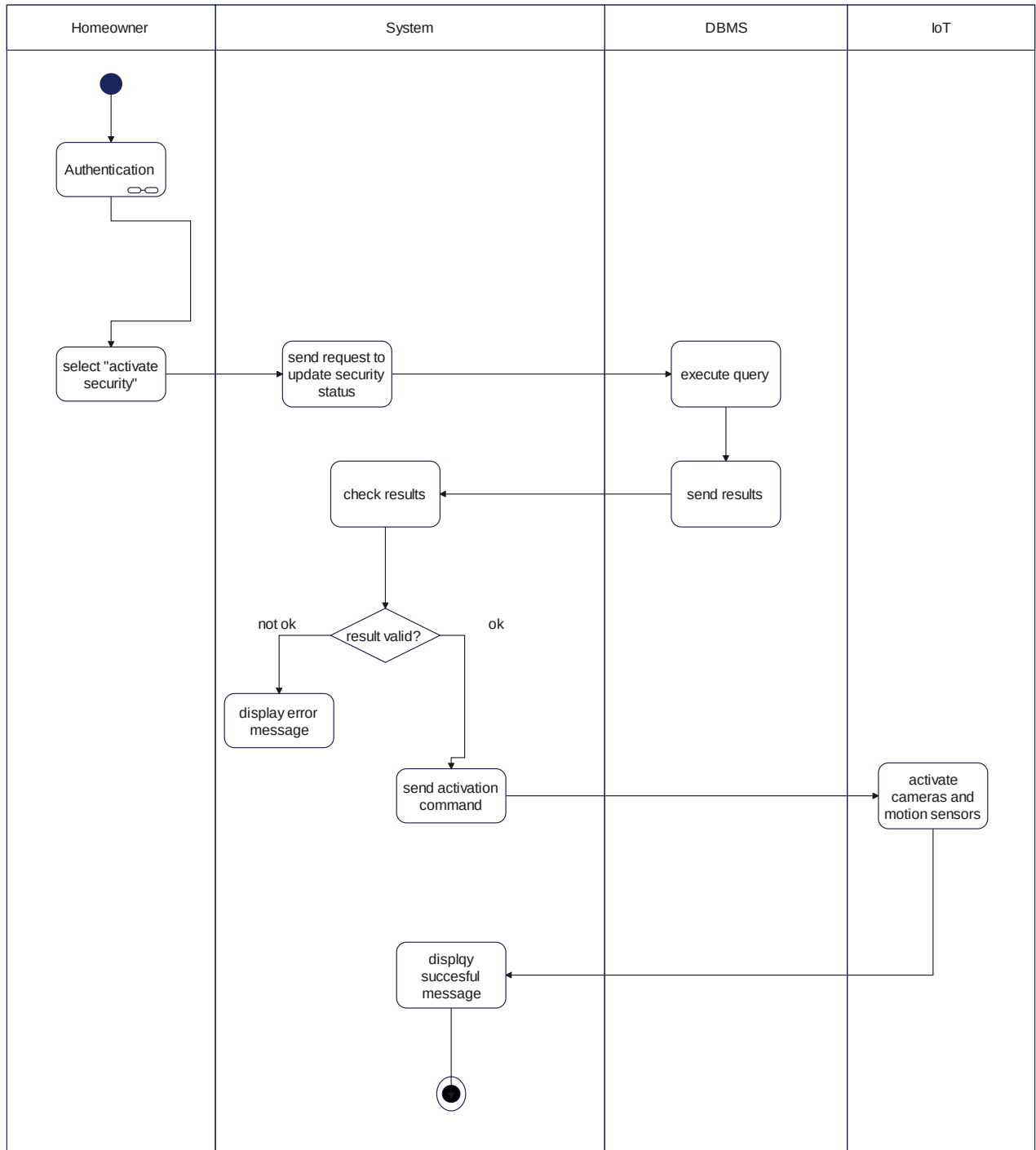


Figure 23:activity diagram for activating security



CONCLUSION

Having reached the end of our analysis, we were asked to illustrate the language and method used during this phase, present the different diagrams of the functional branch. we can then say that this analysis file has allowed us delimit the functional needs of the web application that will be designed and to have a detailed overview of the new system to be set up without notwithstanding we quickly dive into the conception phase



CHAPTER V : CONCEPTION PHASE

INTRODUCTION

- i. Class diagram
- ii. State Machine diagram
- iii. Package diagram

CONCLUSION



INTRODUCTION

The realization of an analysis file is very important for the advancement of the application; it guarantees the reliability of maintenance and use of a software. That said, after analysing our system, we will move on to the design of our future system. To do this we will follow the steps of the technical branch of the 2TUP process, making the diagrams associated with these steps namely the class diagram and the state machine diagram.

A. CAPTURE OF TECHNICAL NEEDS

PHYSICAL ARCHITECTURE

The design of the database heavily relies on its underlying architecture. In this project, we utilized an n-tier architecture, a software design pattern that partitions the entire system into multiple, related but separate layers or tiers. This separation allows for greater flexibility, scalability, and maintainability, as each tier can be independently modified, altered, changed, or replaced without affecting the entire system.

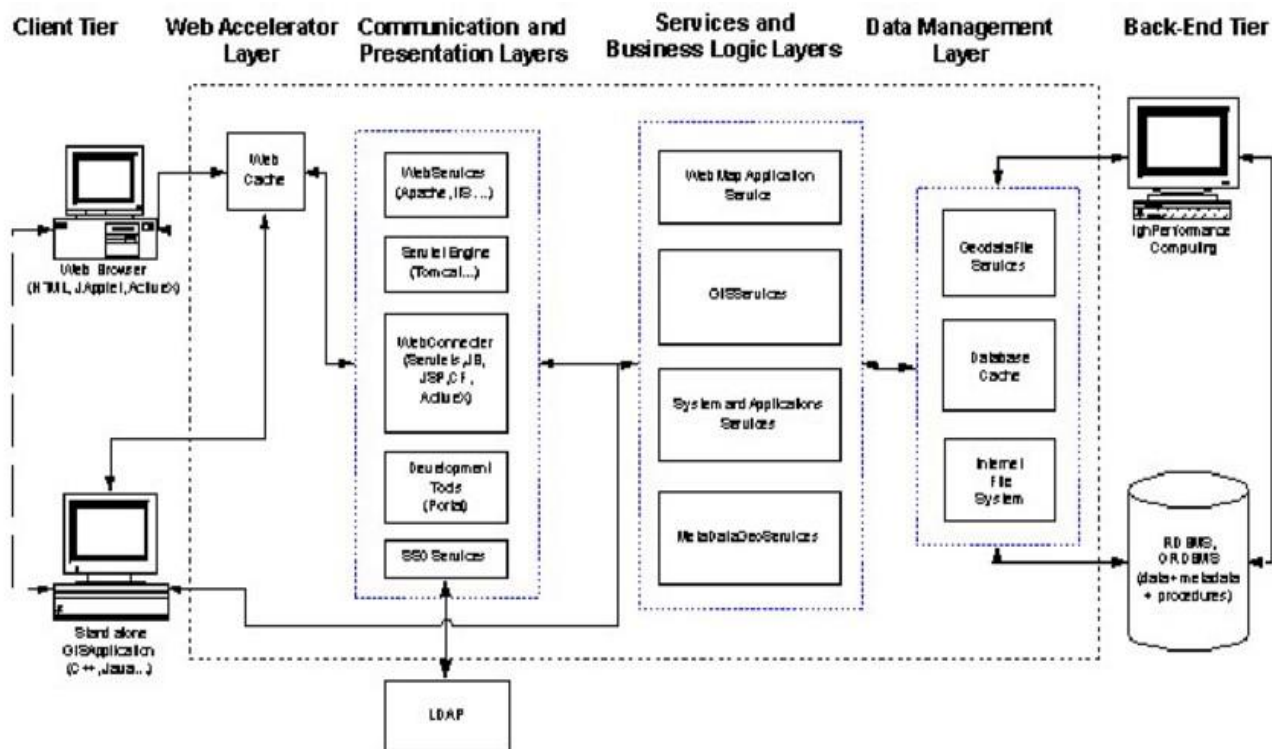


Figure 24: n-tier architecture

Source: <https://www.mindphp.com/images>

LOGICAL ARCHITECTURE

The application logic of our system is based on MVC design pattern for the backend and MVVM for the front end which makes a **hybrid Architecture** a practical way of organizing the different components and classes of an application development project. It defines how the different parts of the system interact with each other, making the application easier to understand, maintain, test, and evolve. A well-defined architecture also facilitates error detection, collaboration between developers, and code reuse.

For the development of the **system**, the **MVVM (Model-View-ViewModel)** design pattern is used to organize the application's software components. It separates the application into three main parts: the **Model**, which manages the application's data and business logic; the **View**, which represents the user interface and displays information to the user; and the **ViewModel**, which acts as an intermediary between the Model and the View by managing presentation logic, application states, and user interactions. This separation of responsibilities makes the application more structured, maintainable, testable, and easier to evolve.

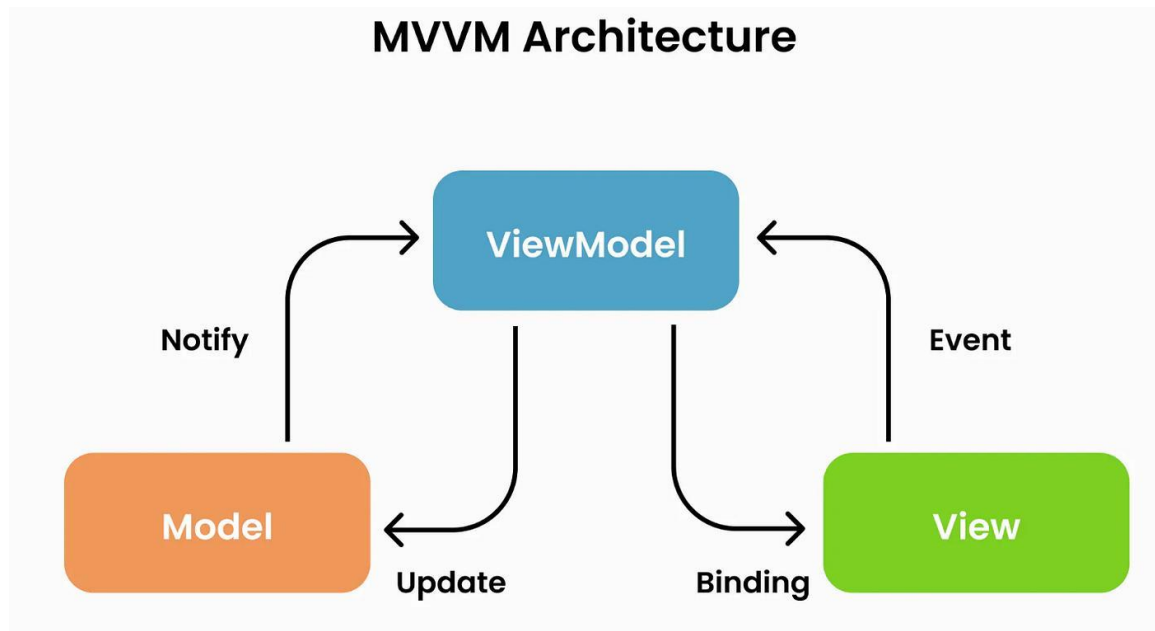


Figure 25: MVVM architecture

https://commons.wikimedia.org/wiki/File:MVVM_architecture.svg?utm_source=chatgpt.com

I. CLASS DIAGRAM

i. Definition

The class diagram expresses the static structure of the system in terms of classes and the relationships between those classes. The interest of the class diagram is to model the entities of the information system. The class diagram is used to represent all the finalized information that is managed by the domain. This information is being structured, that is, it has been grouped into classes.

ii. Formalism

The diagram highlights possible relationships between these classes. A class is a set of elements of the same nature that can be uniquely identified. This diagram has several concepts: class, attribute, identifier, method, access modifier. (Public (+), protected (#), private (-), package (), derivative (/)), relationships and crossplicities. The table below shows the different components of a class.

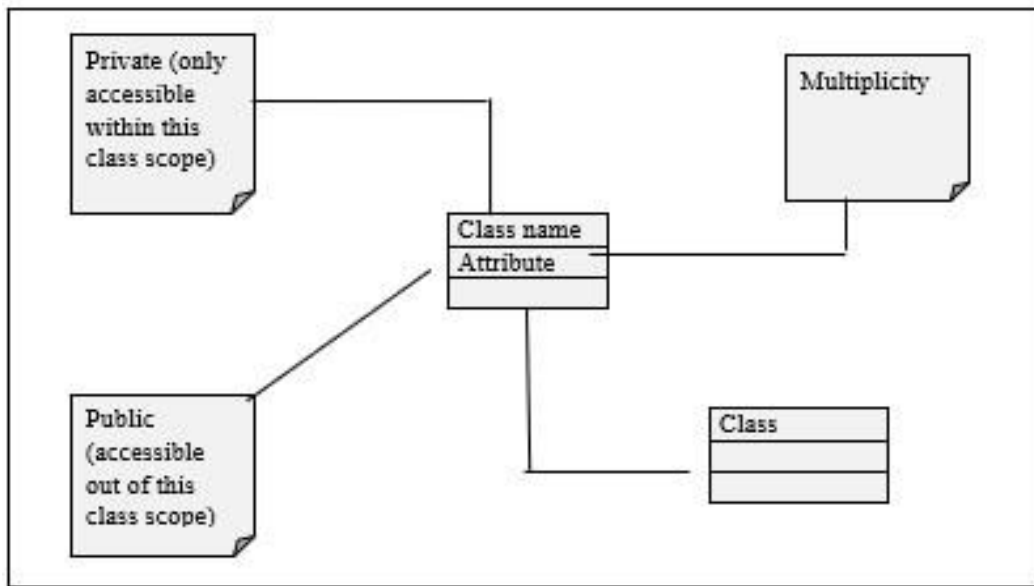
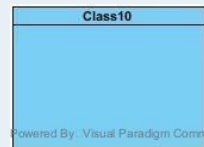
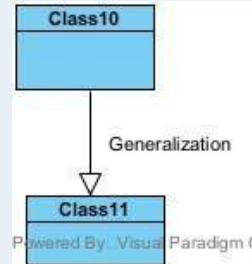
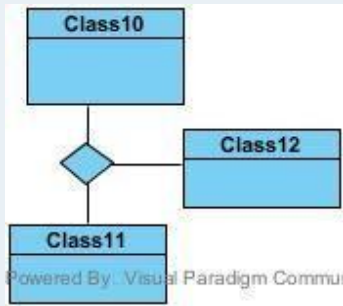
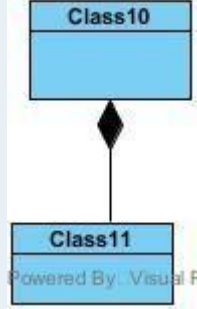


Figure 26: Formalism of class diagram

iii. Components of a Class Diagram

Table 18: Components of a class diagram

Element	Description and main features	Representation
Class	Represents an entity within a system. It is an abstract representation or description of a set of objects in the application domain	

Generalization	Exist between two related classes where one is a specialized form of the other.	 Powered By: Visual Paradigm C
N-ary relation	It is a type of relationship used to model several classes	 Powered By: Visual Paradigm Commu
Composite Association	It is a type of association that represents a part-whole relationship in which one element dies too when the other dies. It is represented by a black diamond	 Powered By: Visual F

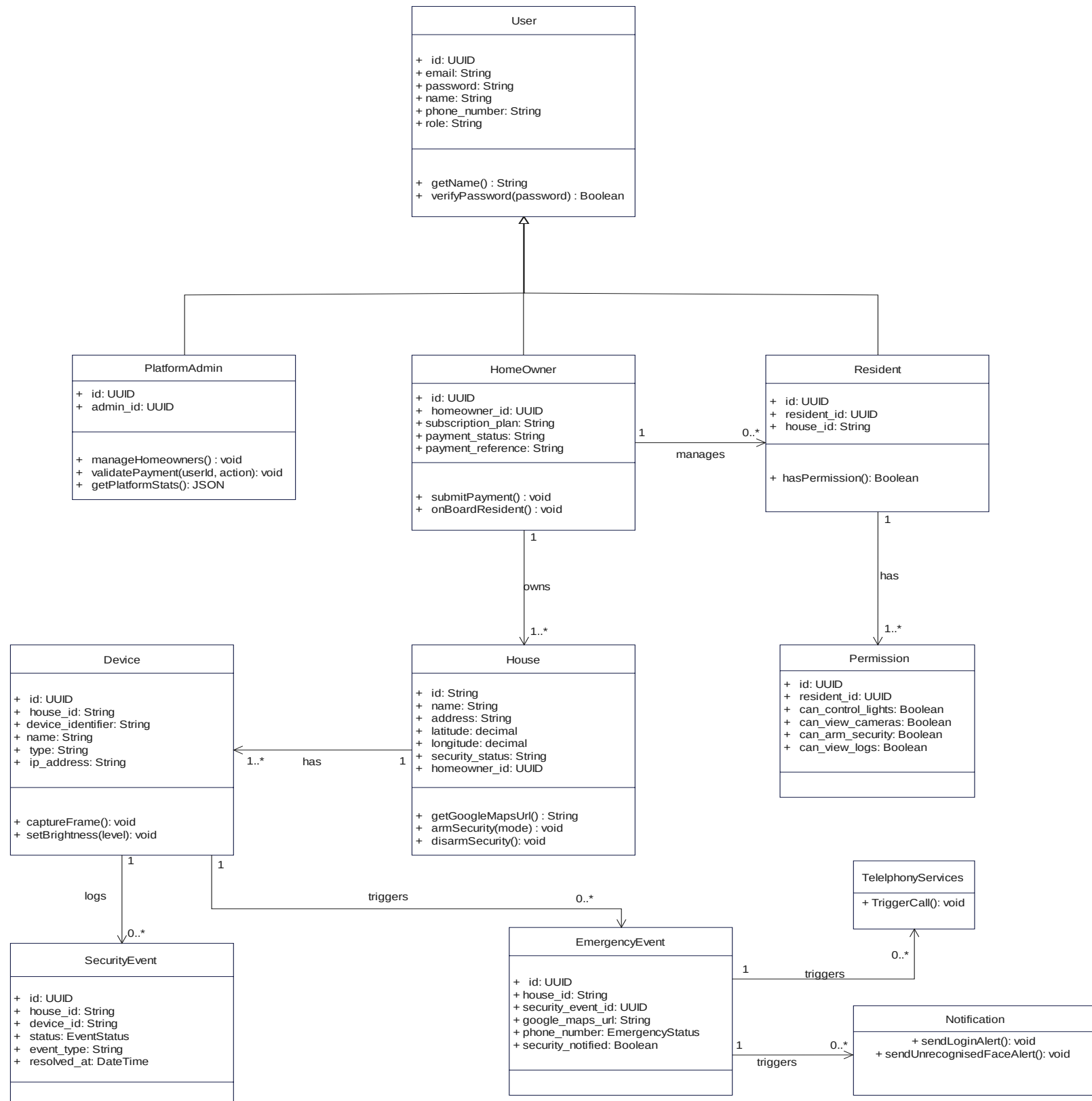


Figure 27: Class diagram for our app

II. STATE MACHINE DIAGRAM

i. Definition

State diagrams are used to document the various modes (states) that a class can take, and the events that cause a state transition. Their role is to present the treatments (operations) that will manage the field studied. They define the sequence of class states, also known as the state chart diagram, which models the dynamic flow of control from the state of a particular object within a system.

ii. Formalism

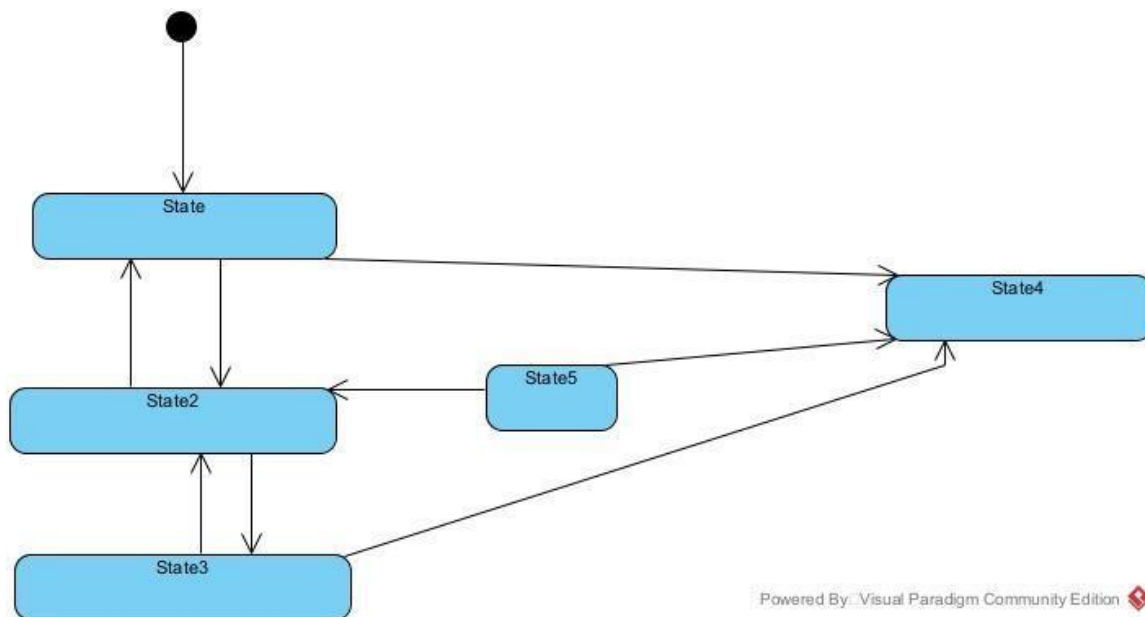
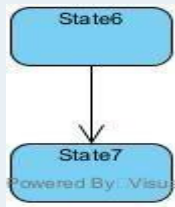


Figure 28: formalism of a state machine diagram

iii. Components of a State Machine Diagram

Table 19: Components of a state machine diagram

Element	Description and features	Representation
Initial State	It represents the final or beginning of state and it is represented by a block filled circle	
Final State	It represents the final state or end of a system. It is represented by a filled circle within another circle	
Transition	it is a change of control from one state to another due to the occurrence of some events. It is represented by an arrow labelled with the event that causes it's change of state.	

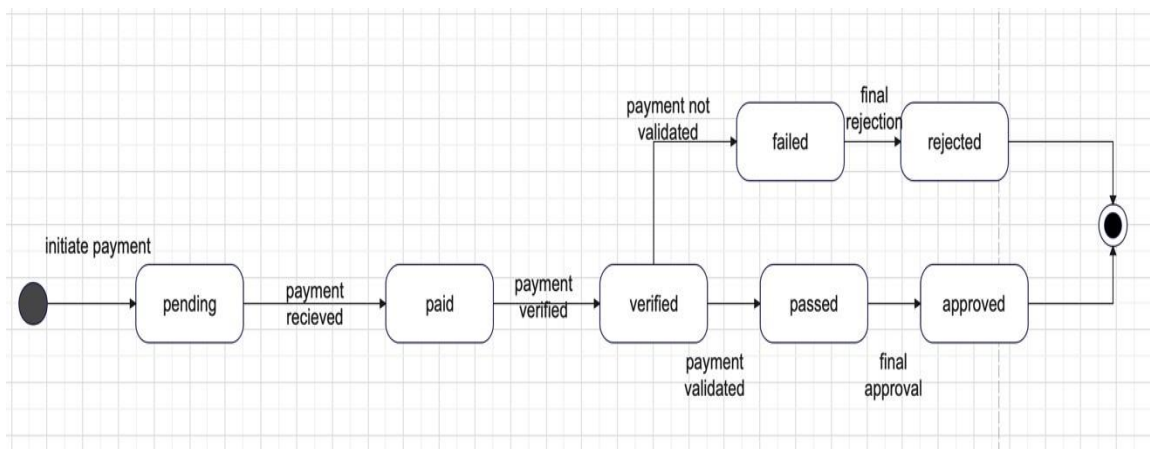
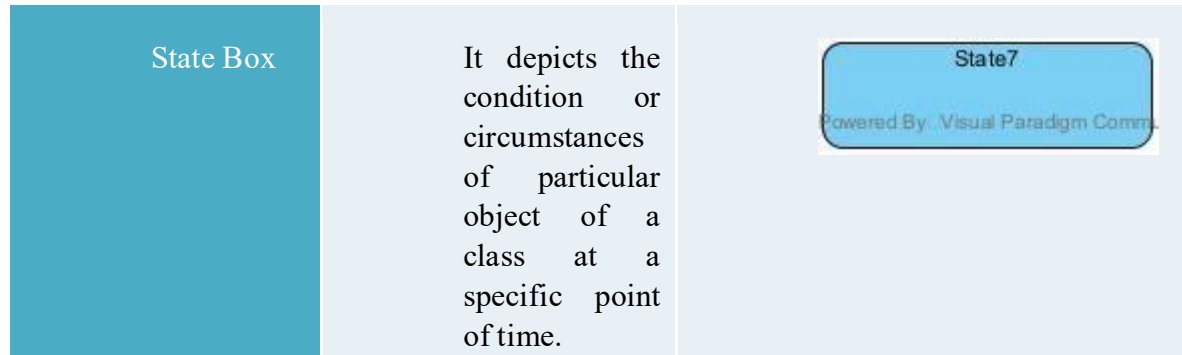


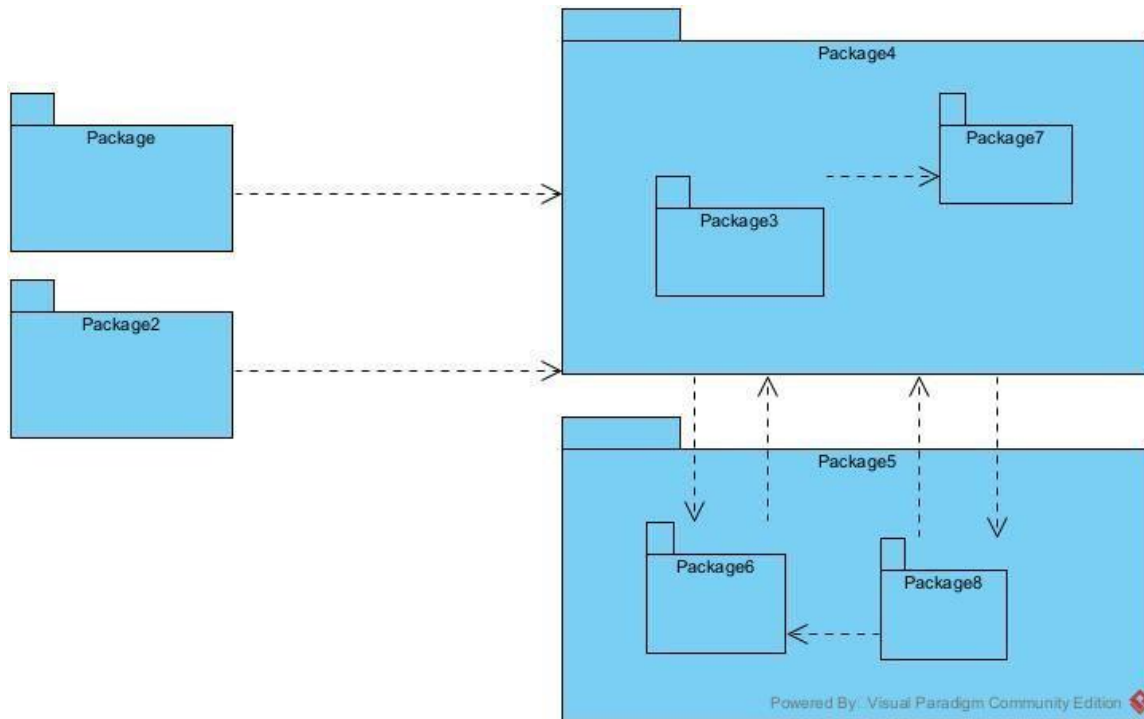
Figure 29: State machine for payment

III. PACKAGE DIAGRAM

i. Definition

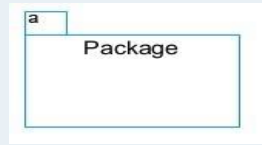
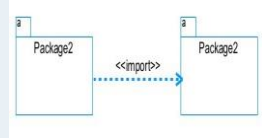
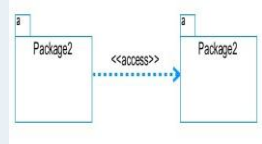
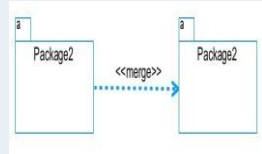
They are structural diagrams used to show the organization and arrangement of various model elements in the form of a package. A package diagram is group of related UML elements such as diagrams, documents, classes, or even other packages. Package diagrams are commonly used to

ii. Formalism



iii. **Component of package diagram**

Element	Description and main features	Representation
---------	-------------------------------	----------------

Package	It is namespace used to group together logically related elements within a system. Each element contains within the package should be a packageable element and have a unique name	
Package import	A relationship Indicate that, functionality has been imported from one package to another.	
Package access	A relationship Indicates that one package requires assistance from the function of another package.	
Package merge	It is a relationship which shows that, the functionality of two packages are combines to a single function.	

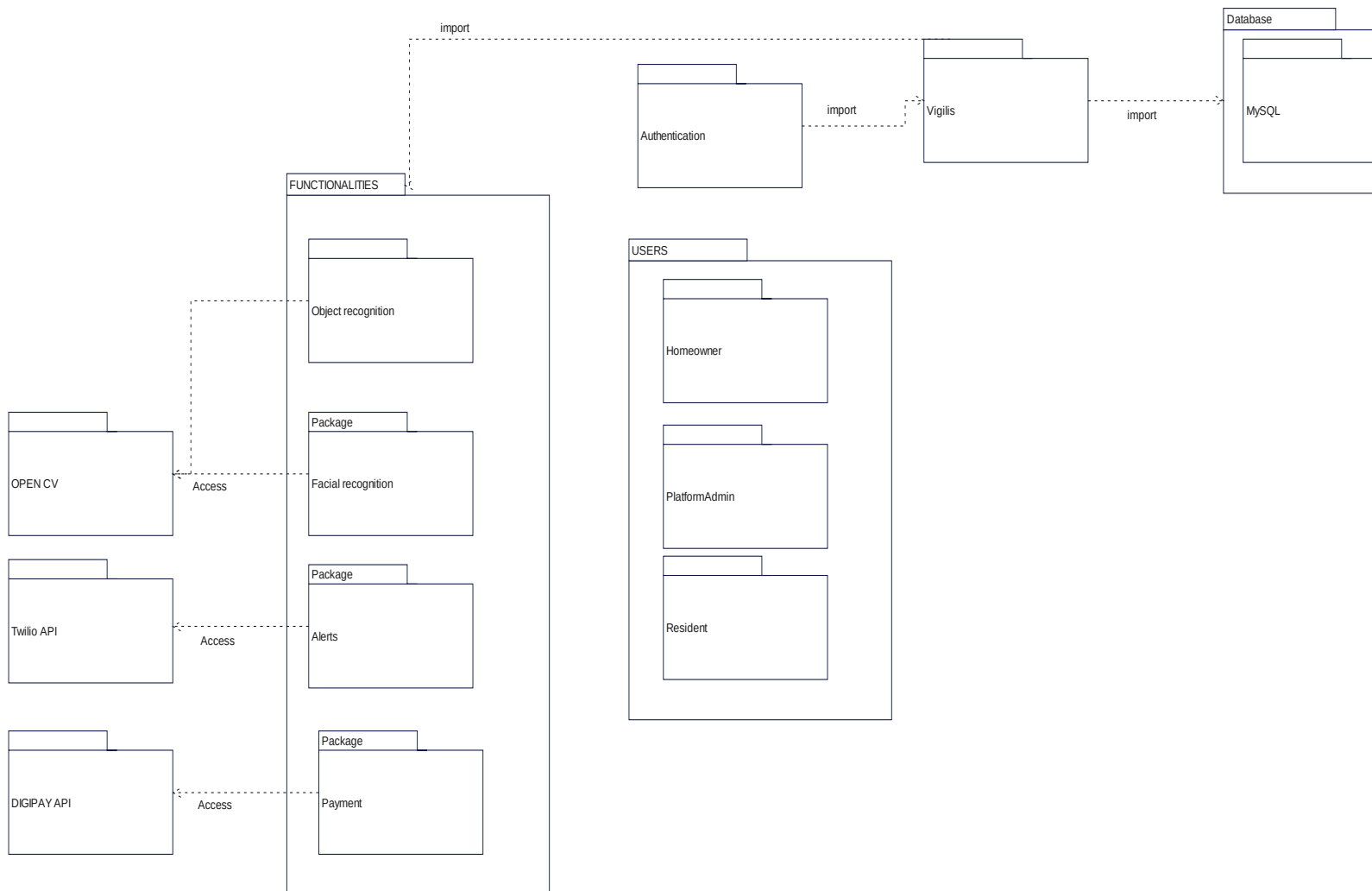


Figure 31: Package diagram for system



CONCLUSION

The conception phase allowed us to bring out the data necessary for the creation of our database and the implementation of our application. The different elements modeled in this part allowed us to have a global view of the different modules of our application; therefore, the next step of our project will be the drafting of the realization file taking into account the different elements modelled above.



CHAPTER VI: REALIZATION PHASE

Preamble

The realization document consists in the implementation of the project in a programming language according to the specifications defined in the previous file. It contains component and deployment diagrams; the testing and development phases. At the end of this part, programming documentation will be produced explaining the architecture of the database and the architecture of our code.

Content :

INTRODUCTION

I. PRELIMINARY CONCEPTION

1. DEPLOYMENT DIAGRAM

2. COMPONENT DIAGRAM

II. TECHNOLOGICAL CHOICES

1. Material Choices

2. Software Choices

3. Technological Choices

III. TECHNOLOGICAL CHOICES

4. Material Choices

5. Software Choices

6. Technological Choices



INTRODUCTION

Design and modelling are important links in the development process of an application because they make it possible to define and specify the different elements constituting the application or system studied. Realization is the culmination of the work carried out upstream because it makes it possible to physically create the various objects that were designed, and to bring the computer system to life. Throughout this phase, we will present the software, physical and hardware architecture of our system, through diagrams such as the component diagram, deployment diagram, physical model and code architecture.

1. DEPLOYMENT DIAGRAM

i. Definition

The deployment diagram models the hardware components used to implement a system and the association between those components. Deployment diagrams can be implemented as early as the design phase to document the physical architecture of the system. As elements used in deployment diagrams we can have, components, as in component diagrams, and nodes, which represent the physical processing resources of the system, and their associations.

ii. Formalism

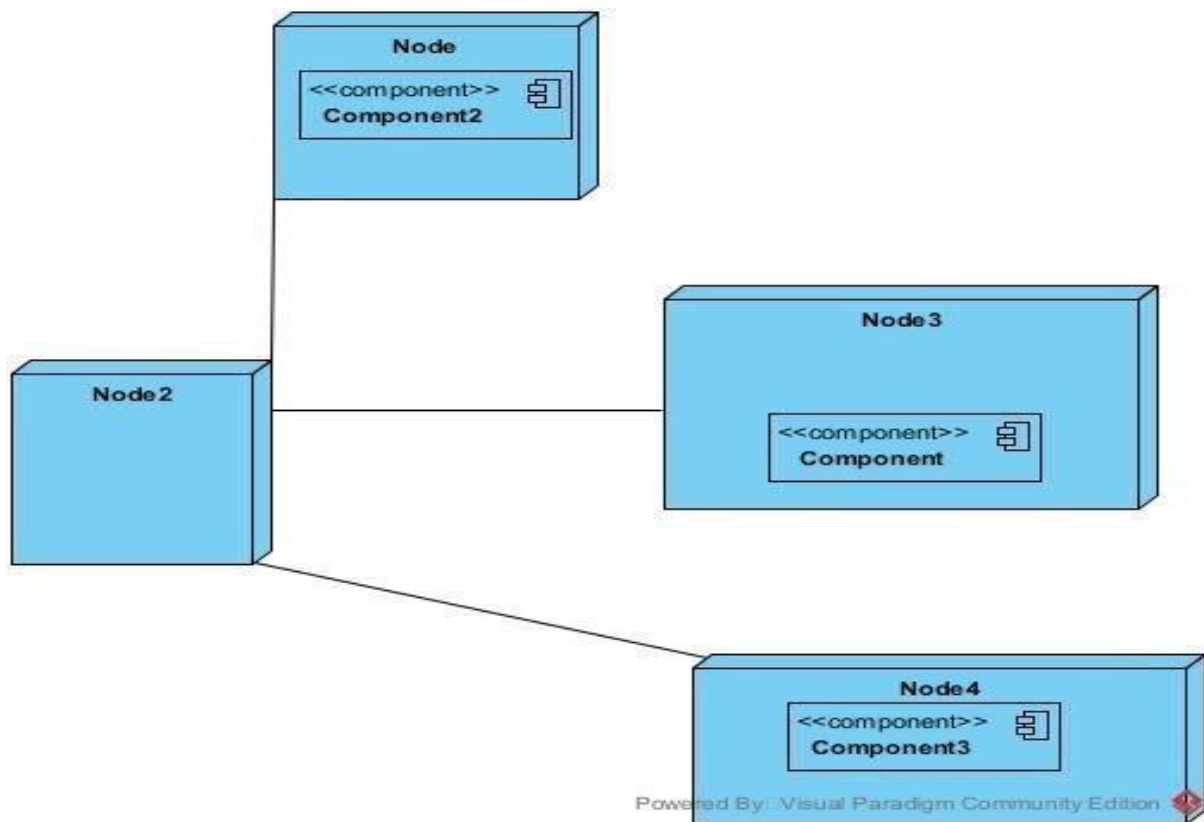
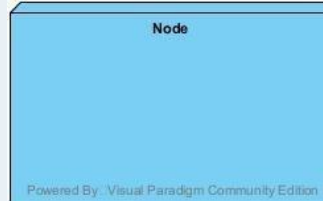
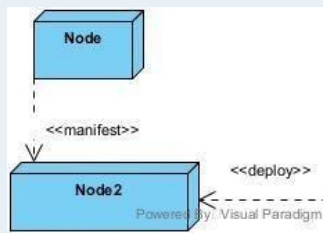


Figure 32: Formalism of deployment diagram

iii. Components of a Deployment Diagram

Table 21: Components of a deployment diagram

Element	Description and main	Representation features
Component	A component represents a software entity of the system	
Node	A node represents a set of hardware elements of the system. This entity or node is represented by a three dimensional cube	
Interface	Represents a declaration of a set of coherent public features and obligations	
Dependency (manifestation deployment)	A relationship between two model elements where changes in one of them will similarly affect the other	

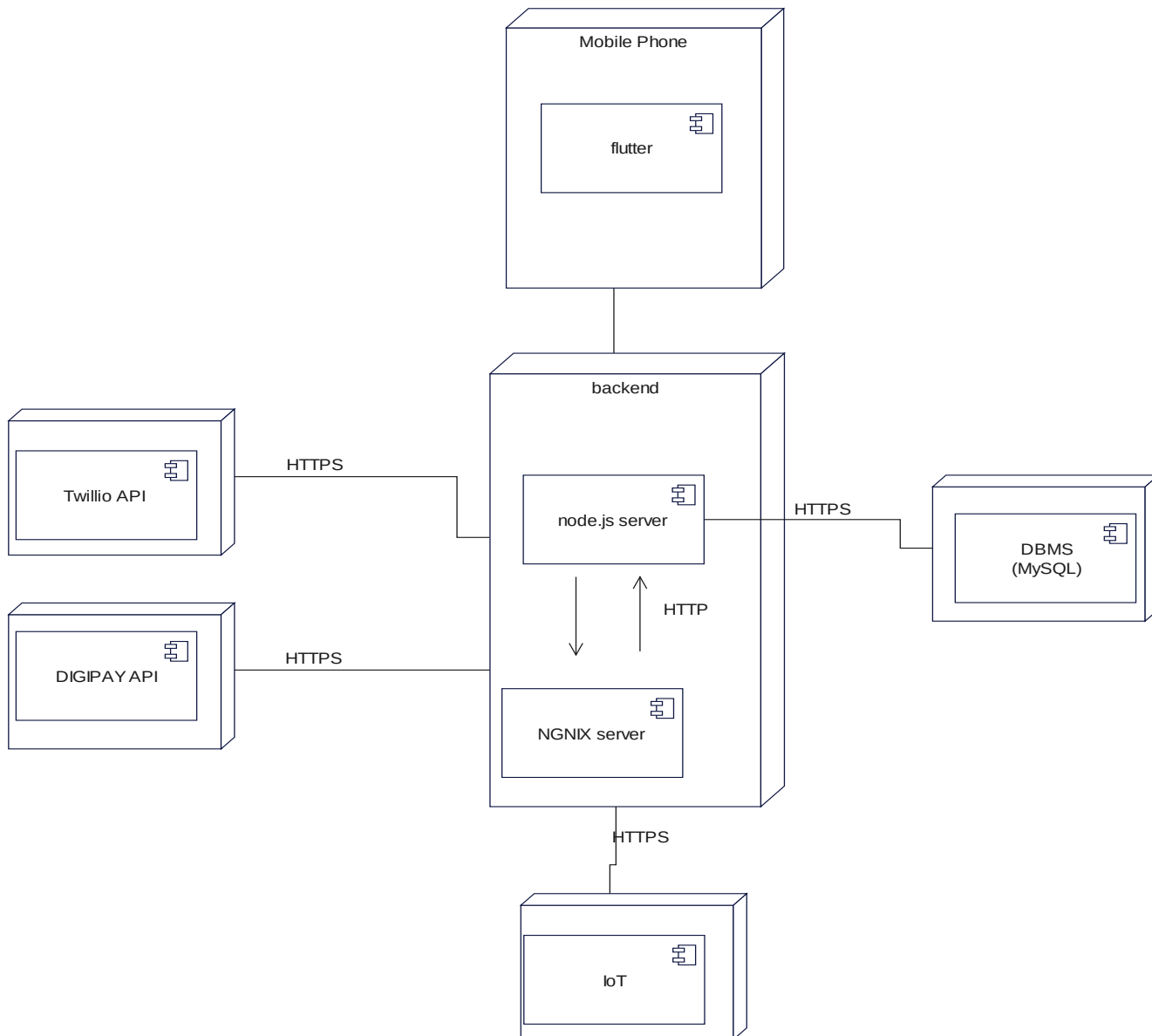


Figure 33: System's deployment diagram

2. COMPONENT DIAGRAM

i. Definition

The component diagram is a UML diagram that provides a graphical representation of dependencies and generalizations between software components, including source code components, binary code components, and executable components.

ii. Formalism

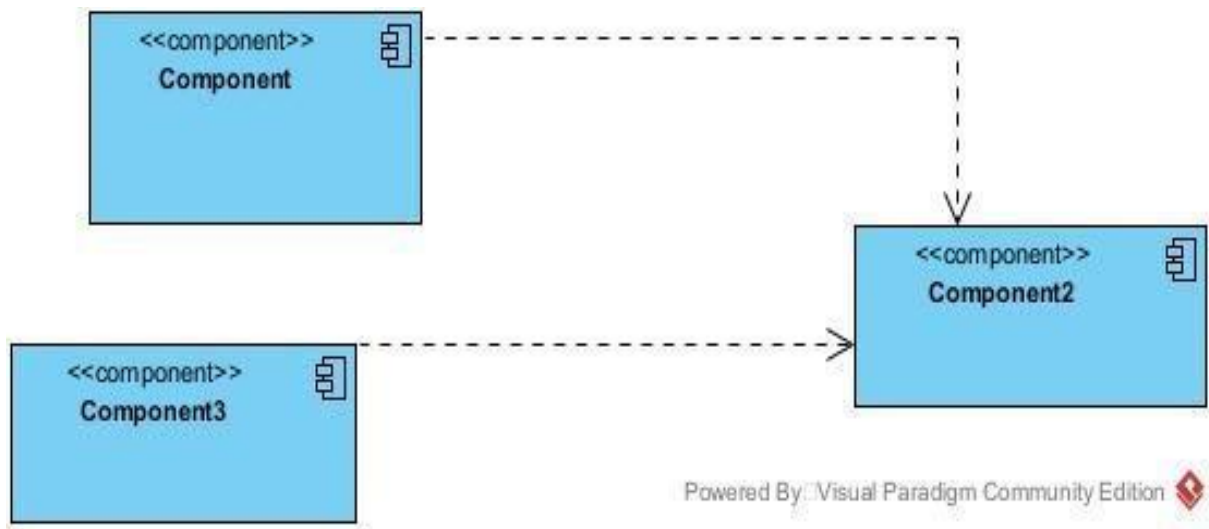
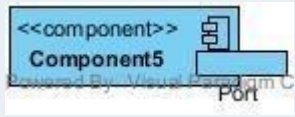
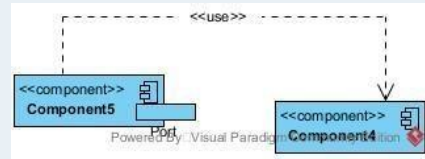


Figure 34: Formalism of component diagram

iii. Components of a Component Diagram

Table 22: Components of a component diagram

Element	Description and features	Representation
Component	Represents a modular part of system that encapsulates its content. In other words it is a part of a system or application.	 Powered By: Visual Paradigm Community Edition

Interface	Represents declaration of a set of coherent pub features and obligations	
Part	Specifies a disti interaction point between that classifier element and its environment	
Usage	A relationship in which one element requires another element for it's full implementation operation	
Instance Specification	It has the capability of being a deployment target in a deployment relationship	

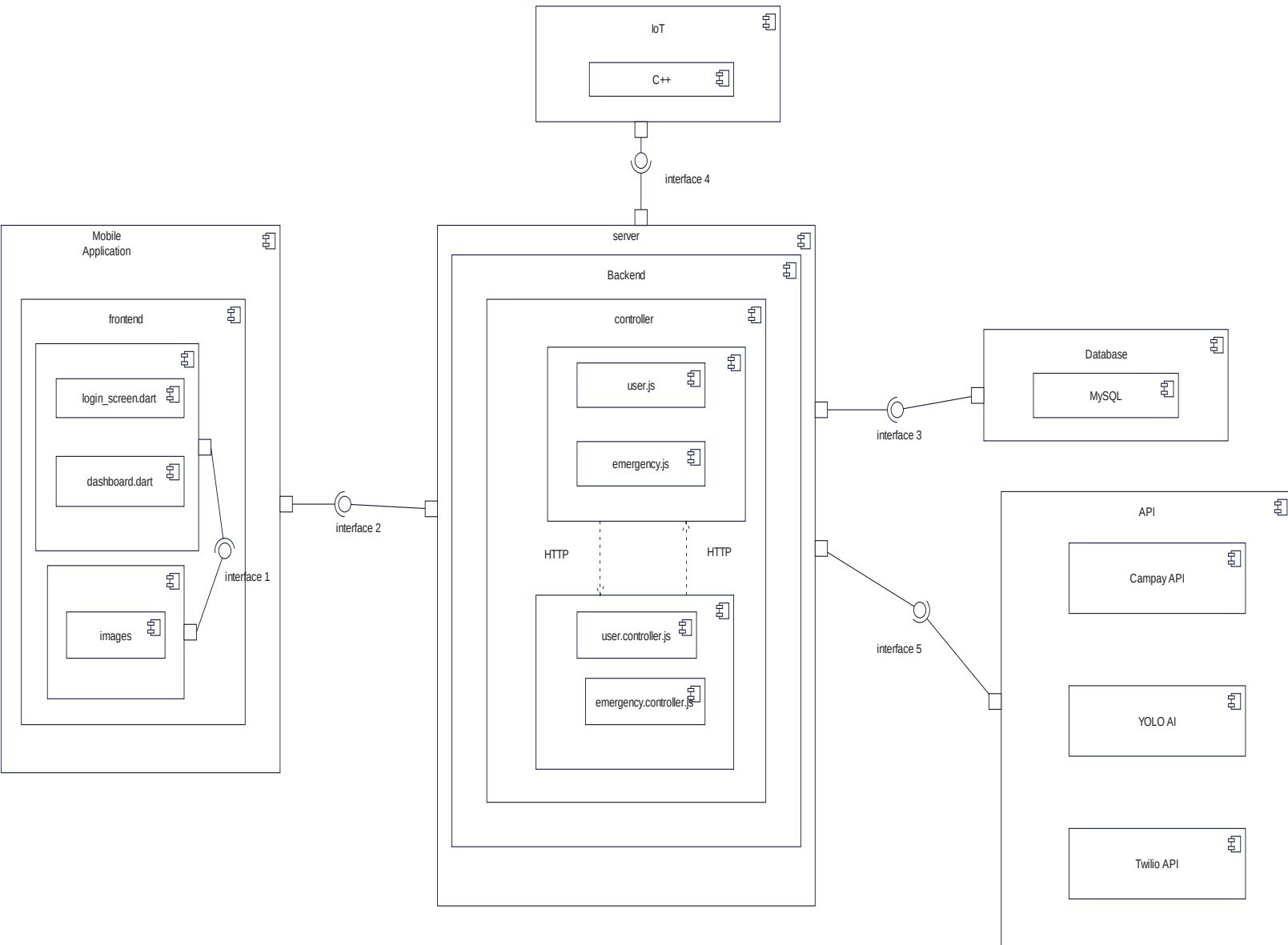


Figure 35: component diagram of the system

TECHNICAL CHOICES

Hardware and Software Materials Used for the Development of the Smart System

Table 23: Hardare material for development

Material	Quantity	Function	Image
Dell LATITUDE E7540	1	The computer was used for the programming of the micro controller, the analysis, research, the design of the system architecture. This was the main component in this project.	
HP computer	1	The computer was used for the development of the mobile application and web application	
iPhone 11 pro max	1	Testing the mobile app functionalities and debugging.	

Table 24: Software used for development

Software	Role	Reason for use	Image
OS WINDOWS 10	The operating system required to run the software is the one on which we have worked.	Windows 10 has a userfriendly interface that is familiar to many users, making it easy to navigate and use.	
Edraw max	A software for the design of diagrams	It facilitates the drawing of UML diagrams	
Visual studio code	Visual Studio Code is a lightweight, open-source code editor developed by Microsoft, supporting multiple programming languages. Using for writing of application code.	Combines tools for version control, debugging, and terminal access in one interface, improving efficiency and reducing the need for multiple applications.	
Github	It is a website and cloud service that helps developers store and manage their code.	Git allows developers to track changes, revert to previous versions, and maintain a history of modifications, ensuring that	

MS Word	MS Word is a widely used word processing application that allows users to create, edit, and format text documents. Using in application report creation.	Offers extensive options for formatting text, images, and layouts, enabling the creation of professional quality documents.	
MS PowerPoint	MS PowerPoint is a presentation software that allows users to create dynamic slideshows comprising text, images, videos, and animations. Able us to create slideshows presentations for application.	Its intuitive interface and templates make it easy for users to create professional presentations quickly, regardless of their technical skill level	
Chrome	Google Chrome is a fast, secure, and user-friendly web browser developed by Google. Used to test our application UI	Chrome is optimized for fast page loading and efficient resource management, providing a smooth browsing experience.	
GanttProject	GanttProject is a free, open-source project management tool that	Allows users to assign tasks, set deadlines, and manage resources	

CONCLUSION

In our realization phase, we set as objective to plane the different aspect of our system by showing how it will be structure and deployed within existing technical architectures. We began by taking into account the technical constraints for our system, after which we proceeded to identify the components of our system, how they are grouped together and how they should be deployed on appropriate deployment targets. We finished this phase by looking at interactions between the various aspects and actors of our system. The next phase of our report is the realization phase where we will look at aspects concerning the implementation of our system.



CHAPTER VII:

FUNCTIONALITY TESTING

Preamble

This phase aims at testing the software or application to verify if the system is working as expected. It involves testing individual functionalities and capabilities of the software to ensure that they work correctly and meets the specified needs.

INTRODUCTION

It is a systematic process that involves evaluating the functionality of a software application or system in different phases or stages of its development lifecycle. This testing approach ensures that each phase of the software development process is thoroughly tested for its intended functionalities and overall performance. Its main goal is to identify and rectify any defects, and bugs during the process.

I. APPLICATION FUNCTIONALITIES

i. Authentication

Authentication is the fundamental functionality of the systems that verifies the identity of users (delivery person, customer, farmer, Admin). It ensures that only authorized individuals or entities can gain access to protected information or perform specific operations.

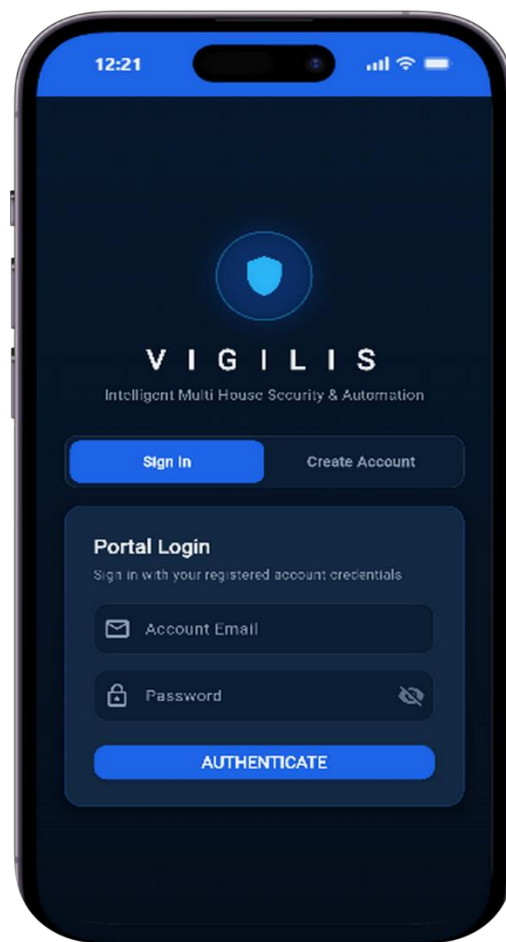


Figure 36: Authentication screen

ii. Add Residents

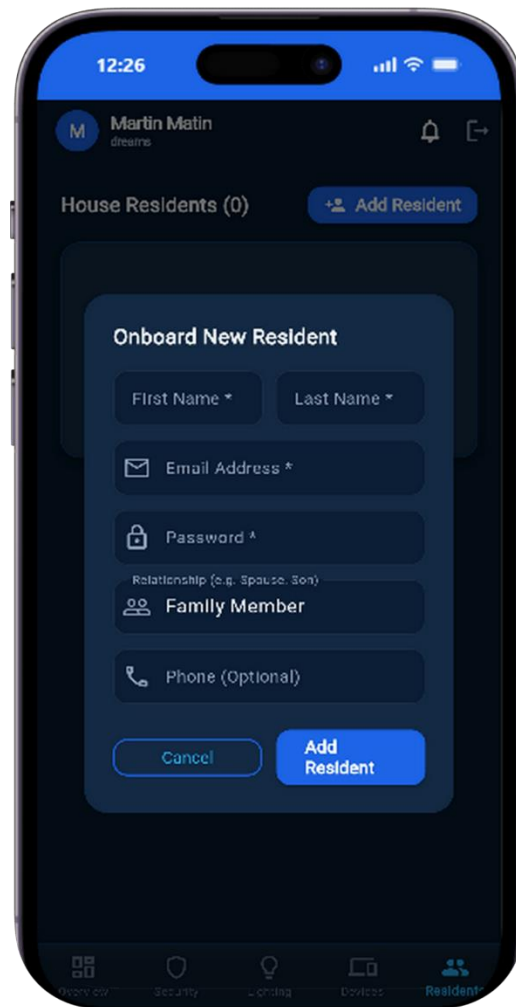


Figure 37: Add resident screen

iii. Smart Lighting Control

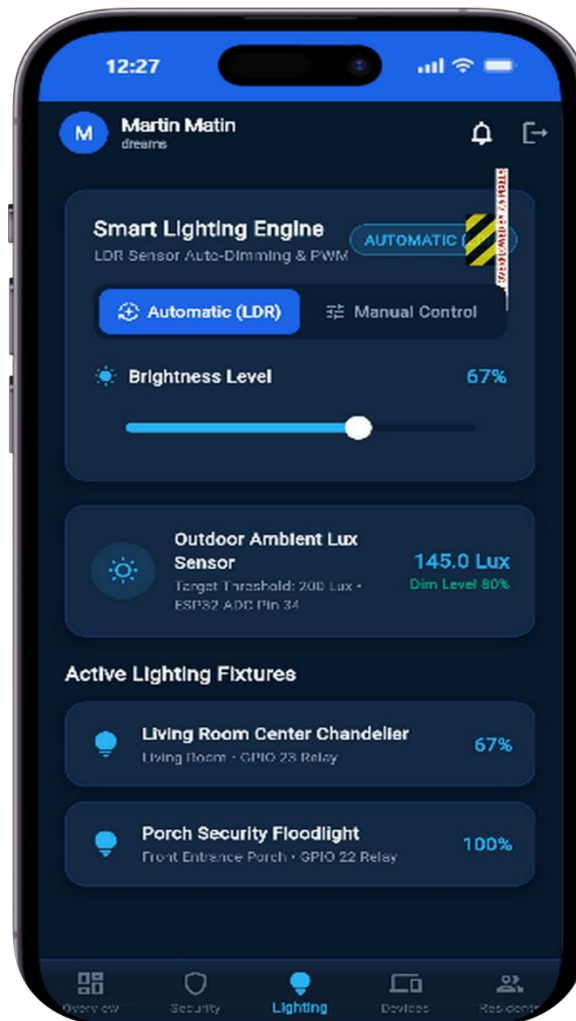


Figure 38: Smart light screen

II. UNIT TESTING

Unit testing refers to testing a function individually, to ensure that it is working properly. Here are some unit test of our solution

i. Login

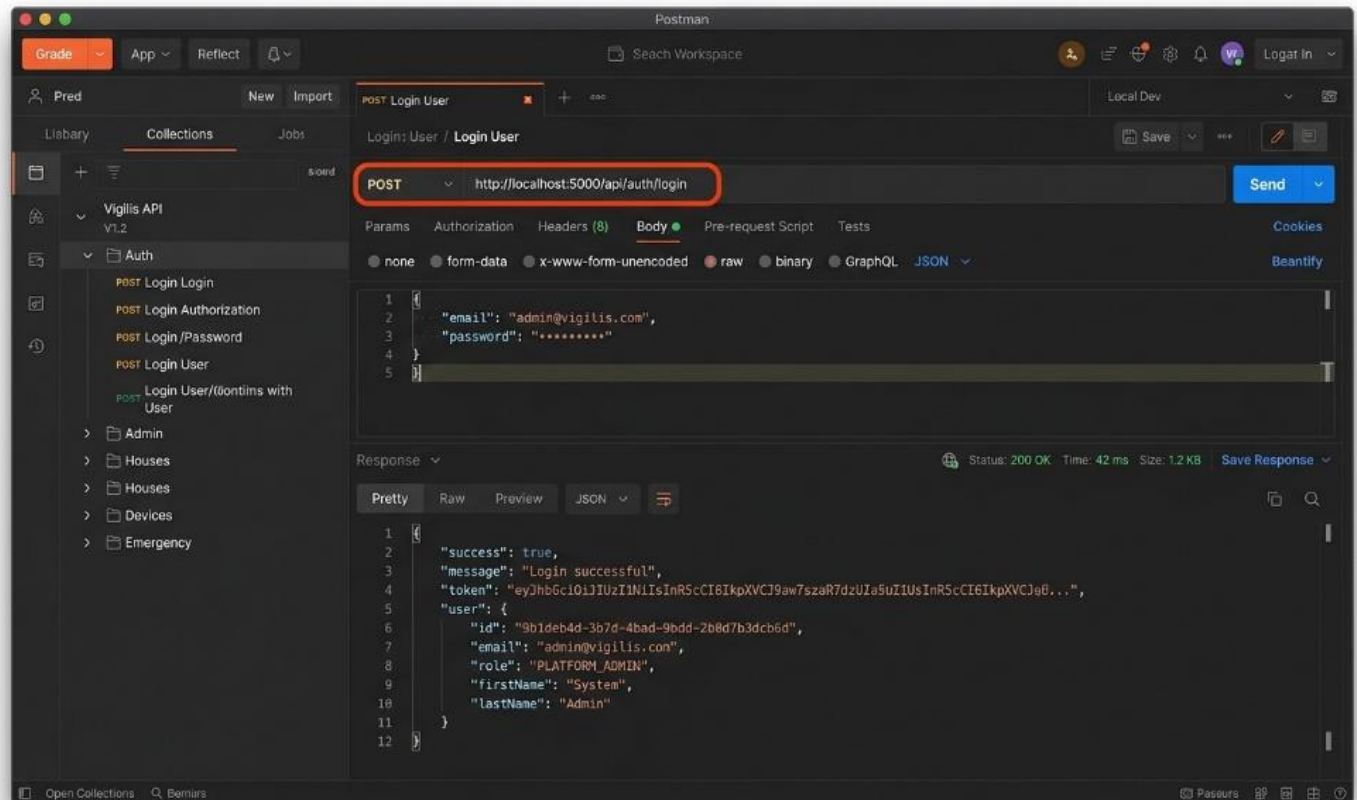


Figure 39: Login test

ii. Add Resident

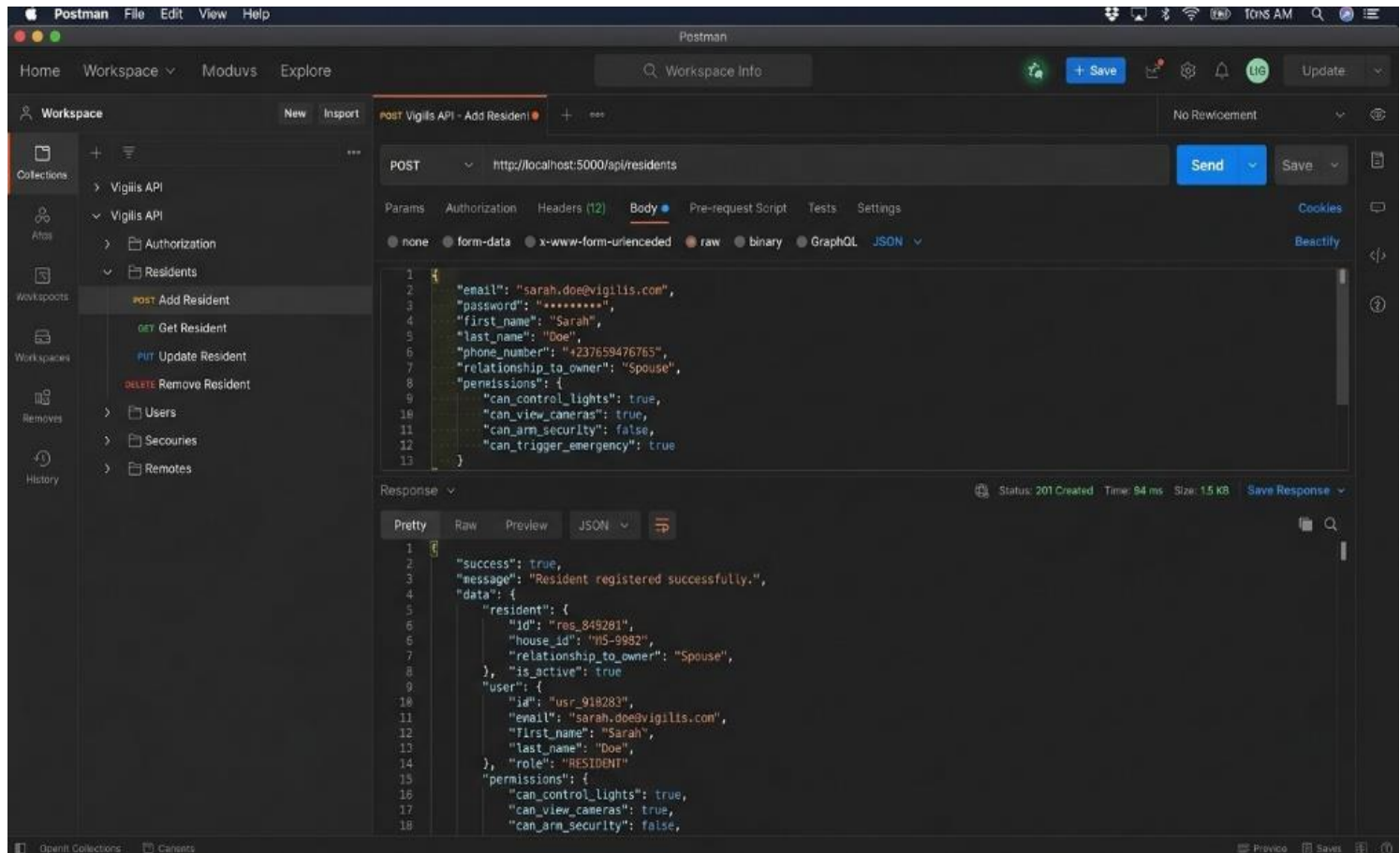


Figure 40: Add resident test

CONCLUSION

Testing is an integral part of the software development life cycle and is performed to verify the correctness, reliability, and robustness of the implemented functionality. the functionality and testing phase are therefore a critical stage in software development where the implemented features are validated, and any issues are identified and resolved.



CHAPTER VIII: INSTALLATION GUIDE AND USER GUIDE

Preamble

A user guide explains how to use a software application in a language that a non- technical person can understand. Thus, it enables the user to easily use the application to familiarize with the software and discover all the its functionalities.

INTRODUCTION

I. DESCRIPTION OF THE APPLICATION

II. INSTALATION GUIDE

III. USER GUIDE

CONCLUSION

INTRODUCTION

The user manual is the document established after the realization of an application, software or platform. It informs on the question "how to use the software, the application or the platform that you have in front of you? ". It is

therefore crucial for us to offer the various users of our platform a guide allowing them to easily perform various operations on our platform.

I. DESCRIPTION OF THE APPLICATION

Vigilis is an intelligent house management and security system designed to improve home safety, comfort, and automation. The system combines intelligent lighting management with real-time security monitoring, allowing homeowners and authorized residents to interact with and monitor their home through a dedicated application.

The system centralizes information related to the house and its connected devices, while providing different users with functionalities according to their assigned roles. Vigilis provides:

- **Intelligent lighting control**, allowing users to switch lights on or off and adjust their brightness according to their needs.
- **Automatic lighting adjustment**, using environmental light information to adapt the intensity of the lighting.
- **Security monitoring**, allowing authorized users to monitor the security status of the house in real time.
- **Camera monitoring**, providing access to connected cameras for observing different areas of the house.
- **Motion detection**, allowing suspicious movements within monitored areas to be detected.
- **Intelligent threat analysis**, using artificial intelligence to analyze security events and distinguish potential threats.
- **Facial recognition**, allowing identified individuals to be recognized and verified by the system.
- **Automatic security alerts**, enabling the system to react when a genuine security threat is identified.
- **Security service notification**, allowing the system to automatically contact the appropriate security service or police when a verified threat requires intervention.

- **User and resident management**, allowing the homeowner to manage accounts and assign appropriate roles and permissions.
- **Multi-house isolation**, ensuring that users can only access the house and connected devices for which they are authorized.

II. ACCESS AND INSTALLATION GUIDE

To use the Vigilis application, the user must have a compatible smartphone or other supported device and install the application. The application must then be connected to the Vigilis system installed in the corresponding house.

After launching the application, the user is presented with the authentication interface. The user must enter their credentials to access the functionalities associated with their account.

Once authentication is successful, the user is redirected to the application's main interface.

III. USER GUIDE

This section presents the different operations that can be performed using the Vigilis application. Each operation can be illustrated with screenshots of the corresponding interfaces in order to help users understand the expected results and correctly manipulate the system.

1. Connecting to the Application

To connect to Vigilis, the user must meet the following prerequisites:

- Have a compatible smartphone or supported device.
- Have the Vigilis application installed.
- Have an active connection to the Vigilis system.
- Have valid login credentials.

After launching the application, the user accesses the login interface and enters their username or email address and password. After successful authentication, the system displays the appropriate dashboard according to the user's role.

2. Login Interface

The login interface allows registered users to securely access the Vigilis application. The user enters their credentials and selects the login option.

If the information provided is correct, access to the application is granted. Otherwise, an error message is displayed and the user is asked to verify their credentials.

3. Dashboard

The dashboard is the main interface of the Vigilis application. It provides users with an overview of the current state of the house and provides quick access to the main functionalities.

Depending on the user's permissions, the dashboard may provide access to:

- Security status;
- Connected cameras;
- Lighting controls;
- Detected movements;
- Security alerts;
- Residents and accounts;
- Connected security devices.

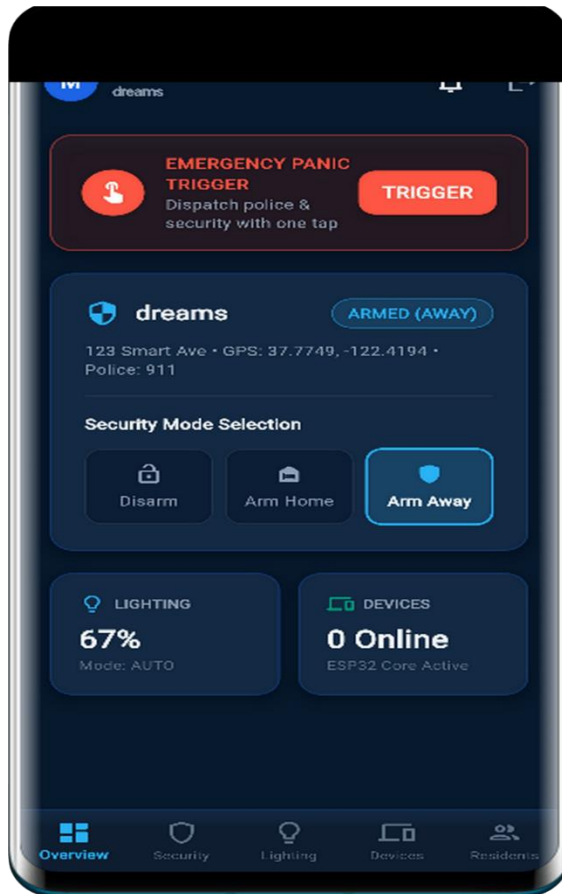


Figure 41:Dashboard screen

4. Manage Lighting

The lighting management interface allows authorized users to control the lights installed in the house.

The user can:

- Turn a light on or off;
- Increase or decrease its brightness;
- View the current lighting state;
- Adjust lighting according to the desired level.

The system can also automatically adjust the lighting intensity according to the detected environmental light conditions.

5. Monitor Security

The security monitoring interface provides an overview of the security status of the house.

Users with the appropriate permissions can view detected movements, security events, alerts, and the current state of connected security devices.

When an abnormal event is detected, the system analyzes the event and determines whether it represents a potential security threat.

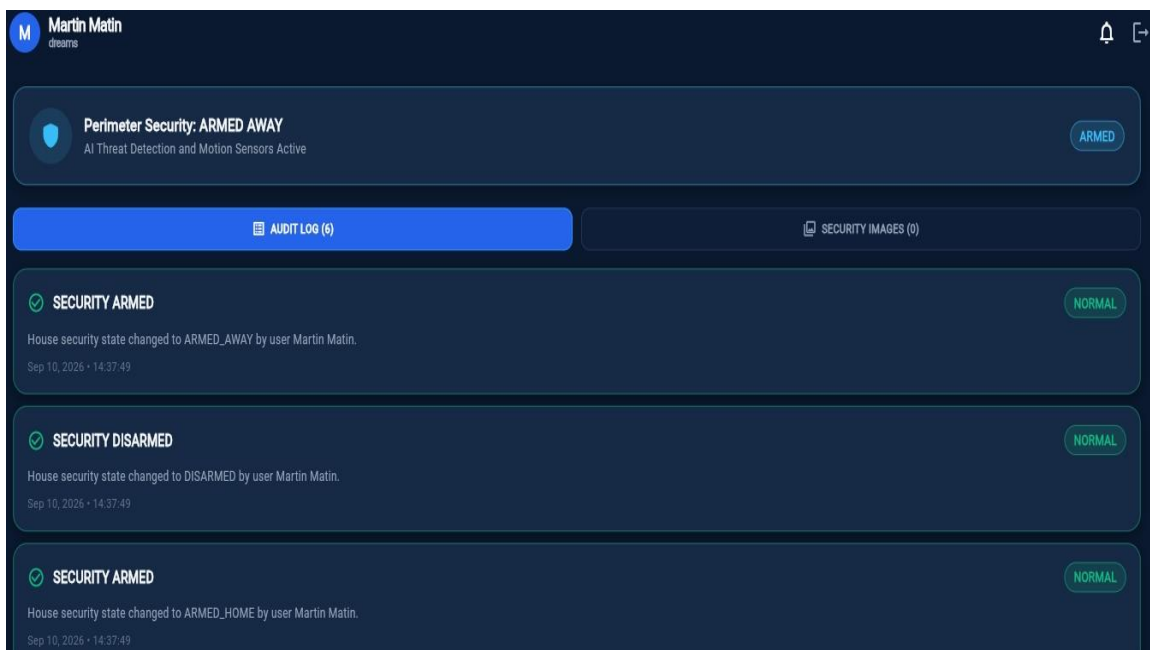


Figure 42: Monitoring security screen

6. Monitor Cameras

The camera monitoring interface allows authorized users to view the cameras connected to their house.

Users can select an available camera and observe the corresponding area in real time. Access to cameras is restricted according to the user's permissions and the house to which the account belongs.

This ensures that users from one house cannot access cameras or security devices belonging to another house.

7. Manage Security Devices

The security device management interface allows authorized users to activate, deactivate, and monitor the security devices installed in the house.

These devices may include motion sensors and other connected security equipment.

The user can verify whether a device is active and monitor its current operating status.

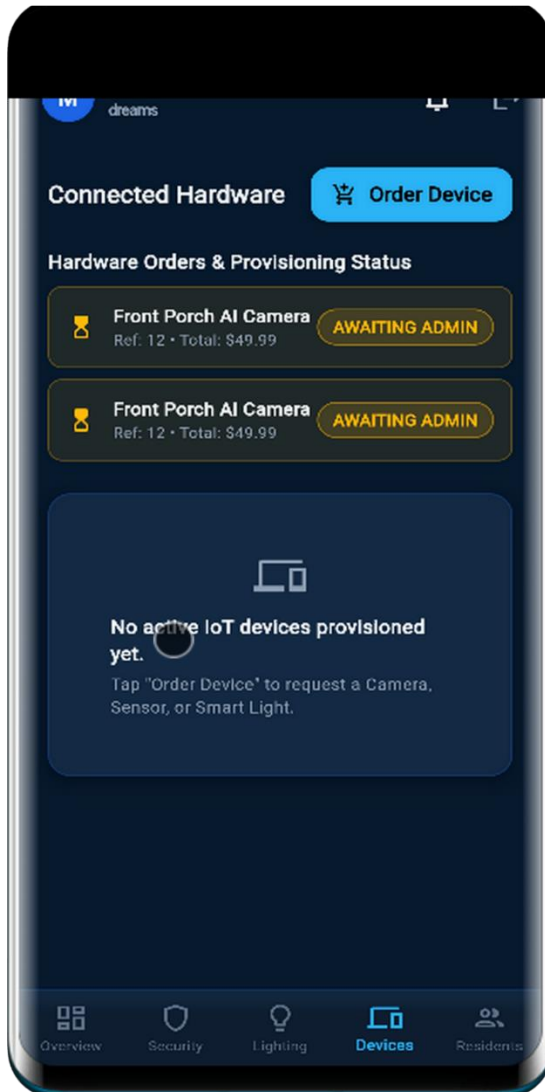


Figure 43: Monitoring devices screen

8. Manage Residents

The homeowner can manage the residents associated with the house.

This functionality allows the homeowner to:

- Add a resident;
- Modify resident information;
- Assign a role;
- Modify permissions;



Design and Implementation of an Intelligent House System Based on Light Intensity Control and Security Monitoring.



- Activate or deactivate a resident account;
- Remove a resident from the house.

The homeowner remains responsible for determining which functionalities each resident can access.

GENERAL CONCLUSION

The development of Vigilis has resulted in an intelligent house management and security system designed to improve both home safety and user comfort. The application provides users with centralized access to essential house functionalities, including intelligent lighting management, security monitoring, camera surveillance, motion detection, and user management.

The integration of artificial intelligence allows the system to analyze security events and identify potential threats, while the automatic notification mechanism provides a rapid response when a genuine threat is verified. The system also incorporates role-based access control and multi-house isolation to ensure that users only access the information and devices for which they are authorized.

Through its different functionalities, Vigilis demonstrates how intelligent technologies can be combined with connected devices to provide a practical, secure, and user-oriented solution for modern homes. The system can also be further enhanced in the future by integrating additional connected devices, improving threat detection, and extending its automation capabilities.

BIBLIOGRAPHY

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley.

Fowler, M. (2003). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.

Jacobson, I., Booch, G., & Rumbaugh, J. (1999). *The Unified Software Development Process*. Addison-Wesley.

Larman, C. (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development* (3rd ed.). Prentice Hall.

Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education Limited.

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

Szeliski, R. (2022). *Computer Vision: Algorithms and Applications* (2nd ed.). Springer.

Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7th ed.). O'Reilly Media.

Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to Build High-Performance Network Programs. *IEEE Internet Computing*, 14(6), 80–83.



ISO/IEC. (2011). *ISO/IEC 19501: Information Technology—Open Distributed Processing—Unified Modeling Language (UML) Version 1.4.2*. International Organization for Standardization.

ISO/IEC/IEEE. (2018). *ISO/IEC/IEEE 29148:2018 — Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*. International Organization for Standardization.

Richardson, L., Amundsen, M., & Ruby, S. (2013). *RESTful Web APIs*. O'Reilly Media.

WEBOGRAPHY

Flutter. (2026). *Flutter Documentation*. Official Flutter documentation.

Website: **docs.flutter.dev**

Flutter. (2026). *Guide to App Architecture — MVVM, Views, ViewModels, Repositories and Services*. Official Flutter documentation.

Website: **docs.flutter.dev/app-architecture**

Node.js. (2026). *Node.js Documentation*. Official Node.js documentation.

Website: **nodejs.org**

Express.js. (2026). *Express — Node.js Web Application Framework*. Official Express.js documentation.

Website: **expressjs.com**

Sequelize. (2026). *Sequelize Documentation*. Official Sequelize documentation.

Website: **sequelize.org**

MySQL. (2026). *MySQL 8.0 Reference Manual*. Oracle/MySQL official documentation.

Website: **dev.mysql.com/doc**



Espressif Systems. (2026). *ESP32 Documentation*. Official Espressif documentation.

Website: **docs.espressif.com**

Ultralytics. (2026). *Ultralytics YOLO Documentation*. Official documentation for YOLO object detection and computer vision.

Website: **docs.ultralytics.com**

Ultralytics. (2026). *Python Usage — YOLO*. Documentation covering the use of YOLO models with Python for prediction, training and computer vision applications.

Website: **docs.ultralytics.com/usage/python**

Ultralytics. (2026). *Model Prediction with Ultralytics YOLO*. Documentation covering image, video, webcam, OpenCV and streaming sources for YOLO inference.

Website: **docs.ultralytics.com/modes/predict**

OpenCV. (2026). *OpenCV Documentation*. Official documentation for computer vision and image/video processing.

Website: **docs.opencv.org**

Dart. (2026). *Dart Documentation*. Official Dart programming language documentation.

Website: **dart.dev**

Microsoft. (2026). *Model-View-ViewModel (MVVM)*. Microsoft Learn documentation on the MVVM architectural pattern.

Website: **learn.microsoft.com**

PlantUML. (2026). *PlantUML Documentation*. Documentation for creating UML diagrams.

Website: **plantuml.com**

MDN Web Docs. (2026). *JavaScript Documentation*. Reference documentation for JavaScript and web development technologies.

Website: **developer.mozilla.org**



Stack Overflow. (2026). *Stack Overflow — Developer Questions and Answers*. Community resource consulted for technical problem solving and implementation issues.

Website: **stackoverflow.com**

ANNEXES